SLB-OBDH2-DOC-v0.9

OBDH 2.0 Documentation

OBDH 2.0 Documentation
SpaceLab, Universidade Federal de Santa Catarina, Florianópolis - Brazil

November 2021

OBDH 2.0 Documentation
November, 2021

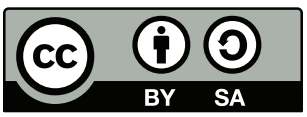
Project Chief:
Eduardo Augusto Bezerra

Authors:
Gabriel Mariano Marcelino
André Martins Pio de Mattos
Yan Castro Azeredo

Contributing Authors:

Revision Control:

Version	Author(s)	Changes	Date
0.1	Gabriel M. Marcelino	Document creation	10/2019
0.5	Gabriel M. Marcelino	First stable hardware	08/2020
0.6	A. M. P. Mattos, G. M. Marcelino, Y. A. Azeredo	First hardware integration	04/2021
0.7	A. M. P. Mattos, G. M. Marcelino, Y. A. Azeredo	General firmware and hardware improvements	06/2021
0.8	Gabriel M. Marcelino	Adding the FRAM memory	10/2021
0.9	Gabriel M. Marcelino	TBC	TBC



List of Figures

1.1 3D view of the OBDH 2.0 PCB. 1

2.1 Product tree of the OBDH 2.0 module. 4
2.2 OBDH 2.0 Block diagram. 5
2.3 System layers. 5
2.4 OBDH 2.0 operation flowchart 7
2.5 OBDH's 2.0 boot sequence. 8
2.6 Antenna deploy routine 9
2.7 Telecommand's processing flowchart 10
2.8 Transmit/Receive flowchart 11
2.9 Data path diagram. 12
2.10 Available status LEDs. 12

3.1 Top side of the PCB. 13
3.2 Bottom side of the PCB. 14
3.3 Side view of the PCB. 14
3.4 Interfaces diagram. 15
3.5 Bottom view of PC-104 and simplified labels 16
3.6 Antenna module conectors. 17
3.7 Programmer (P1 and P2) and jumper (P6) connectors. 18
3.8 Dedicated UART debug connectors (P7). 18
3.9 Samtec FSI-110-03-G-D-AD connector. 19
3.10 Daughterboard connector (P3). 19
3.11 Recommended shape and size of the daughterboard. 20
3.12 Illustrative daughterboard integration. 20
3.13 Microcontroller internal diagram. 21
3.14 Microcontroller pinout positions. 22
3.15 External watchdog timer circuit. 25
3.16 External memory circuit. 25
3.17 FRAM memory circuit. 26
3.18 I2C buffer circuit. 26
3.19 RS-485 transceiver circuit. 27

4.1 Product tree of the firmware of the OBDH 2.0 module. 30
4.2 Diagram of the used HMAC scheme. 42

5.1 Additional GPIOs and I2C channel 1. 48
5.2 Additional I2C channel 0. 48
5.3 Additional GPIO and SPI channel. 48

6.1 Firmware initialization on PuTTy. 50

B.1 Top view of the OBDH 2.0 v0.5 board. 60
B.2 Bottom view of the OBDH 2.0 v0.5 board. 61
B.3 Log messages during the first boot. 62
B.4 I^2C port 0 test. 62
B.5 I^2C port 1 test. 63
B.6 I^2C port 2 test. 63
B.7 64
B.8 Current sensor fix. 64
B.9 Log messages with the read values from the current sensor. 65
B.10 NOR memory SPI test. 66

C.1 Top view of the OBDH 2.0 v0.7 board. 68
C.2 Bottom view of the OBDH 2.0 v0.7 board. 68
C.3 Log messages during the first boot. 69
C.4 Setup of the I2C port tests. 70
C.5 Waveform of the I2C port 0. 71
C.6 Waveform of the I2C port 1. 71
C.7 Waveform of the I2C port 2. 72
C.8 Test results of the NOR flash memory. 73
C.9 Test results of the FRAM memory. 74

List of Tables

3.1 Boards interfaces. . . . 15
3.2 PC-104 connector pinout. . . . 16
3.3 Antenna module connectors pinout. . . . 17
3.4 Programmer header connector pinout. . . . 18
3.5 Programmer picoblade connector pinout. . . . 18
3.6 Daughterboard connector pinout. . . . 20
3.7 Microcontroller features summary. . . . 21
3.8 USCI configuration. . . . 21
3.9 Microcontroller pinout and assignments. . . . 24

4.1 External libraries and dependencies of the firmware. . . . 29
4.2 Firmware tasks. . . . 31
4.3 Variables and parameters of the OBDH 2.0. . . . 34
4.4 EDC information packet. . . . 35
4.5 OBDH data packet. . . . 36
4.6 EPS data packet. . . . 37
4.7 TTC data packet. . . . 38
4.8 Antenna data packet. . . . 38
4.9 SBCD PTT data packet. . . . 39
4.10 Telecommunication packets and their content. . . . 40
4.11 IDs of the satellite. . . . 41
4.12 Enter hibernation telecommand. . . . 42
4.13 Leave hibernation telecommand. . . . 42
4.14 Leave hibernation telecommand. . . . 43
4.15 Ping telecommand. . . . 44
4.16 Ping telecommand answer. . . . 44
4.17 Message broadcast telecommand. . . . 45
4.18 FreeRTOS main configuration parameters. . . . 46

5.1 Additional PC104 inferfaces. . . . 48

A.1 Downlink packets. . . . 55
A.2 Uplink packets. . . . 58

Contents

List of Figures vi

List of Tables vii

Nomenclature vii

1 Introduction 1

2 System Overview 3
- 2.1 Product tree . . . 3
- 2.2 Block Diagram . . . 3
- 2.3 System Layers . . . 4
- 2.4 Operation . . . 5
  - 2.4.1 Execution Flow . . . 6
  - 2.4.2 Data Flow . . . 6
  - 2.4.3 Status LEDs . . . 6

3 Hardware 13
- 3.1 Interfaces . . . 14
- 3.2 External Connectors . . . 14
  - 3.2.1 PC-104 . . . 15
  - 3.2.2 Antenna Module . . . 17
  - 3.2.3 Programmer and Debug . . . 17
  - 3.2.4 Daughterboard . . . 18
- 3.3 Microcontroller . . . 19
  - 3.3.1 Interfaces Configuration . . . 21
  - 3.3.2 Clocks Configuration . . . 21
  - 3.3.3 Pinout . . . 22
- 3.4 External Watchdog . . . 24
- 3.5 Non-Volatile Memories . . . 25
  - 3.5.1 Flash NOR . . . 25
  - 3.5.2 FRAM . . . 25
- 3.6 I2C Buffers . . . 26
- 3.7 RS-485 Transceiver . . . 26
- 3.8 Voltage and Current Sensors . . . 27

4 Firmware 29
- 4.1 Product tree . . . 29
- 4.2 Dependencies . . . 29

4.3 Tasks . . . 29
4.3.1 Antenna deployment . . . 29
4.3.2 Antenna reading . . . 29
4.3.3 Data log . . . 29
4.3.4 General Telemetry . . . 31
4.3.5 EDC reading . . . 31
4.3.6 EPS reading . . . 31
4.3.7 Heartbeat . . . 31
4.3.8 Housekeeping . . . 31
4.3.9 Mission Manager . . . 32
4.3.10 Payload X reading . . . 32
4.3.11 Position Determination . . . 32
4.3.12 Read sensors . . . 32
4.3.13 Startup (boot) . . . 32
4.3.14 System reset . . . 32
4.3.15 Telecommand processing . . . 32
4.3.16 Time control . . . 32
4.3.17 TTC reading . . . 32
4.3.18 Watchdog reset . . . 33
4.4 Variables and Parameters . . . 33
4.5 Telemetry . . . 34
4.5.1 EDC Information . . . 34
4.5.2 OBDH Information . . . 34
4.5.3 EPS Information . . . 35
4.5.4 TTC Information . . . 35
4.5.5 Antenna Information . . . 35
4.5.6 SBCD Packets . . . 35
4.6 Telecommands . . . 36
4.6.1 Authentication . . . 41
4.6.2 Enter hibernation . . . 41
4.6.3 Leave hibernation . . . 41
4.6.4 Activate Module . . . 42
4.6.5 Deactivate Module . . . 43
4.6.6 Activate Payload . . . 43
4.6.7 Deactivate Payload . . . 43
4.6.8 Erase Memory . . . 43
4.6.9 Force Reset . . . 43
4.6.10 Get Payload Data . . . 43
4.6.11 Set Parameter . . . 44
4.6.12 Get Parameter . . . 44
4.6.13 Ping . . . 44
4.6.14 Broadcast message . . . 44
4.6.15 Data request . . . 45
4.6.16 Update TLE . . . 45
4.6.17 Transmit packet . . . 45
4.7 Operating System . . . 45
4.8 Hardware Abstraction Layer (HAL) . . . 46

5 Board Assembly **47**
5.1 Development Model 47
5.1.1 Debug and programming connectors 47
5.1.2 Status leds 47
5.2 Flight Model 47
5.3 Custom Configuration 47

6 Usage Instructions **49**
6.1 Powering the Board 49
6.2 Log Messages 49
6.3 Daughterboards Installation 49

References **51**

Appendices **53**

A Telecommunication Packets **53**
A.1 Downlink 53
A.2 Uplink 55

B Test Report of v0.5 Version **59**
B.1 Visual Inspection 59
B.2 Firmware Programming 59
B.3 Communication Busses 60
B.4 Sensors 61
B.4.1 Input Voltage 61
B.4.2 Input Current 63
B.5 Peripherals 65
B.5.1 NOR Flash Memory 65
B.6 Conclusion 66

C Test Report of v0.7 Version **67**
C.1 Visual Inspection 67
C.2 Firmware Programming 68
C.3 Communication Busses 69
C.4 Sensors 70
C.4.1 Input Voltage 70
C.4.2 Input Current 72
C.5 Peripherals 72
C.5.1 NOR Flash Memory 72
C.5.2 FRAM Memory 73
C.6 Conclusion 73

CHAPTER 1

Introduction

The OBDH 2.0 is an On-Board Computer (OBC) module designed for nanosatellites. It is one of the service modules developed for the GOLDS-UFSC CubeSat Mission [1]. The module is responsible for synchronizing actions and the data flow between other modules (i.e., power module, communication module, payloads) and the Earth segment. It packs the generated data into data frames and transmits it back to Earth through a communication module or stores it on non-volatile memory for later retrieval. Commands sent from the Earth segment to the CubeSat are received by radio transceivers located in the communication module and redirected to the OBDH 2.0, which takes the appropriate action or forward the commands to the target module.

The module is a direct upgrade from the OBDH of FloripaSat-1 [2], which grants a flight heritage rating. The improvements focus on providing a cleaner and more generic implementation compared to the previous version, along with more reliability in software and hardware implementations and adaptations for the new mission requirements. The whole project, including source and documentation files, is available freely on a GitHub repository [3] under the GPLv3 license.

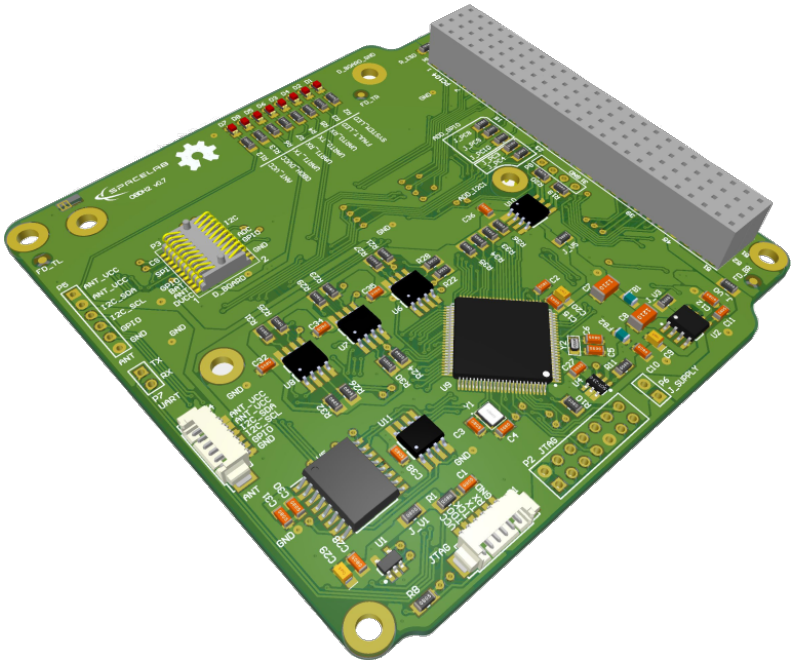


Figure 1.1: 3D view of the OBDH 2.0 PCB.

CHAPTER 2

System Overview

The board has an MSP430 low-power microcontroller that runs the firmware application and several other peripherals for extended operation and physical interfaces (i.e., non-volatile memory, watchdog timer, service modules, payloads interfaces, daughterboard interface, and current monitor). The microcontroller manages the other sub-modules within the board using serial communication buses, synchronizes actions, handles communication with the ground segment, and manages the data flow. The programming language used is C, and the firmware was developed using the Code Composer Studio IDE (a.k.a. CCS) for compiling, programming and testing. The module has many tasks over distinct protocols and time requirements, such as interfacing peripherals and other MCUs. To improve predictability, a Real-Time Operating System (RTOS) is used to ensure that the deadlines are observed, even under a faulty situation in a routine. The RTOS chosen is the FreeRTOS (v10.0.0), since it is designed for embedded systems applications and was already validated in space applications. The firmware architecture follows an abstraction layer scheme to facilitate higher-level implementations and allow more portability across different hardware platforms.

2.1 Product tree

The product tree of the OBDH 2.0 module is available in Figure 2.1.

2.2 Block Diagram

Figure 2.2 presents a simplified view of the module subsystems and interfaces. The microcontroller has a programming JTAG and 6 communication buses, divided into 3 different protocols (I2C, SPI, and UART), that is shared between all the peripherals and external interfaces. Besides these channels, there are GPIO connections for various functions, from control ports to status pins. There is a non-volatile memory device to store the satellite data frames and critical status indicators. Some buffers and transceivers allow secure and proper communication with external modules. A watchdog timer with a voltage monitor and a current sensor is attached to the system to improve the overall reliability and generate essential housekeeping data. There is a generic daughterboard interface for extending the module capabilities with an auxiliary application board. Also, a UART debug interface is directly connected to the microcontroller. More details and descriptions about these components and interfaces are provided in chapter 3.

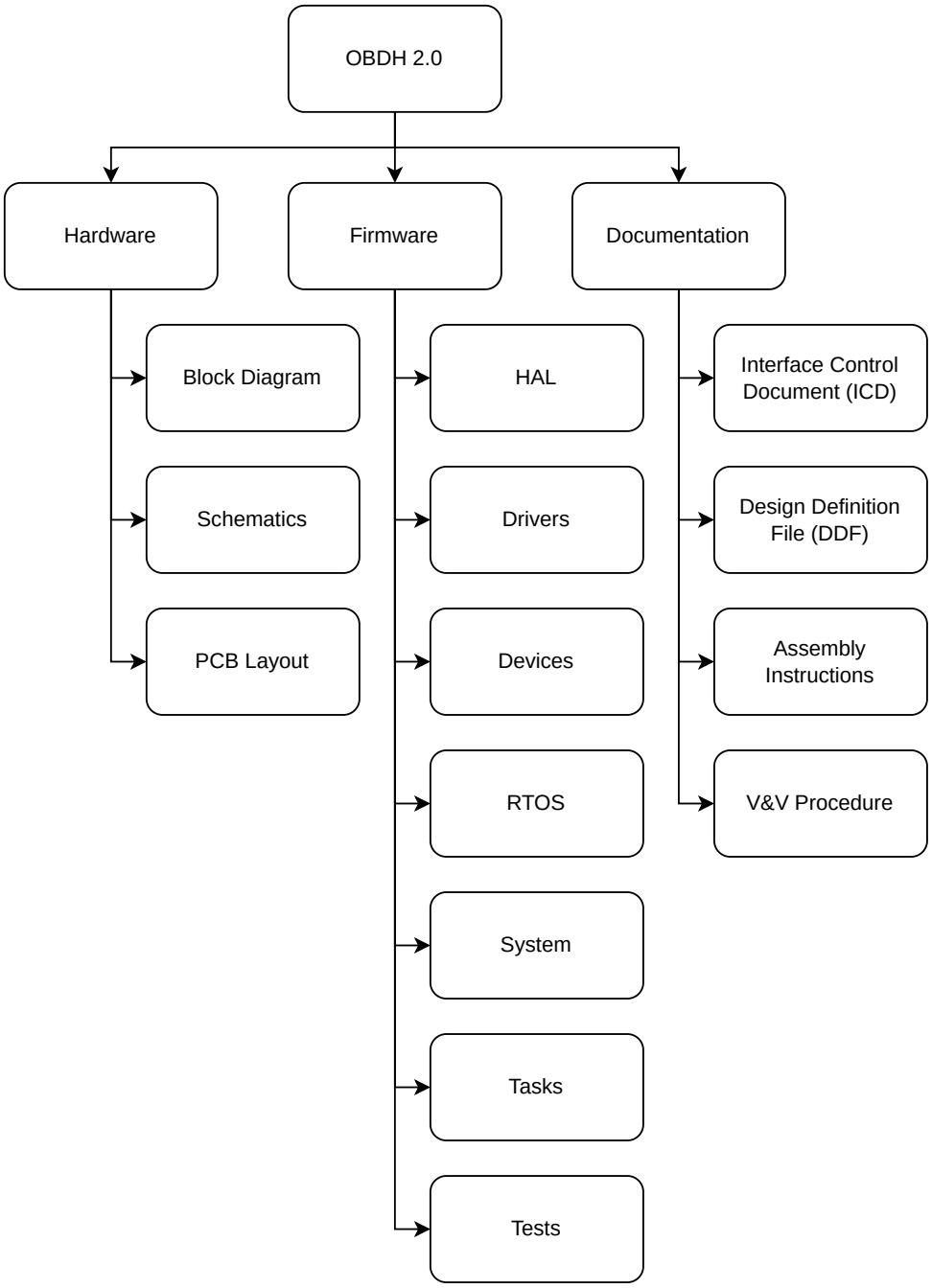


Figure 2.1: Product tree of the OBDH 2.0 module.

2.3 System Layers

As mentioned, the system is divided into abstraction layers to favor high-level firmware implementations. The Figure 2.3 shows this scheme, composed of third-party drivers at the lowest layer above the hardware, the operating system as the base building block of the module, the devices handling implementation, and the application tasks in the highest layer. More details are provided in chapter 4.

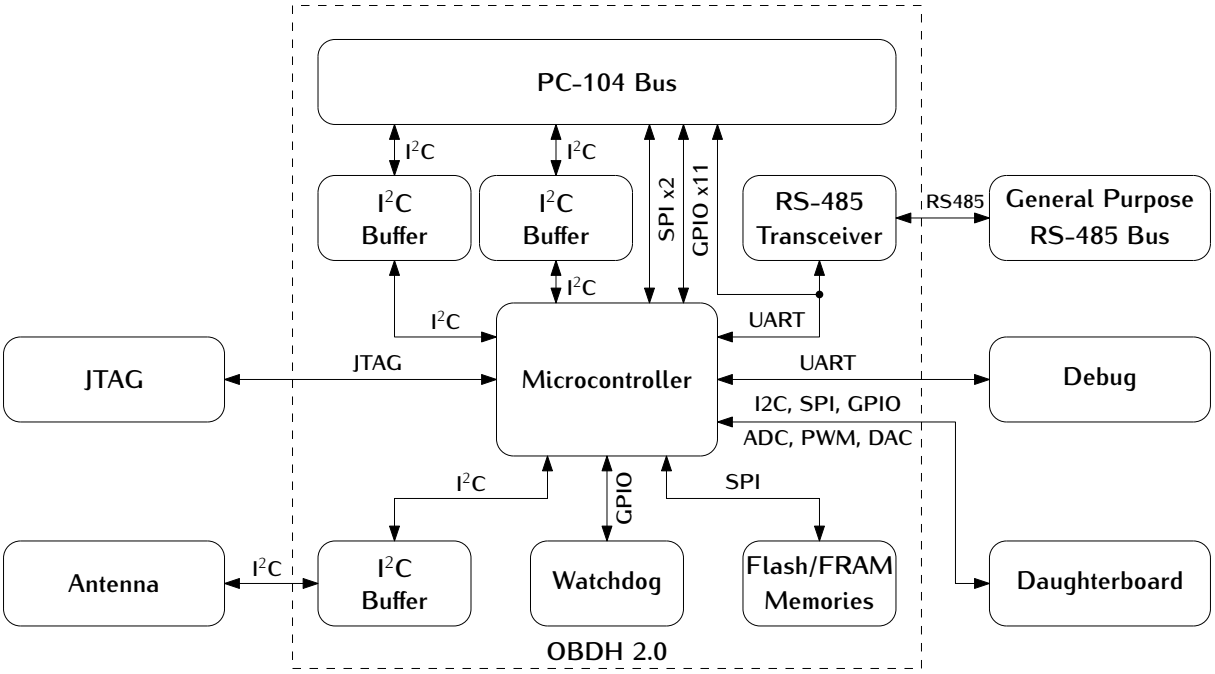


Figure 2.2: OBDH 2.0 Block diagram.

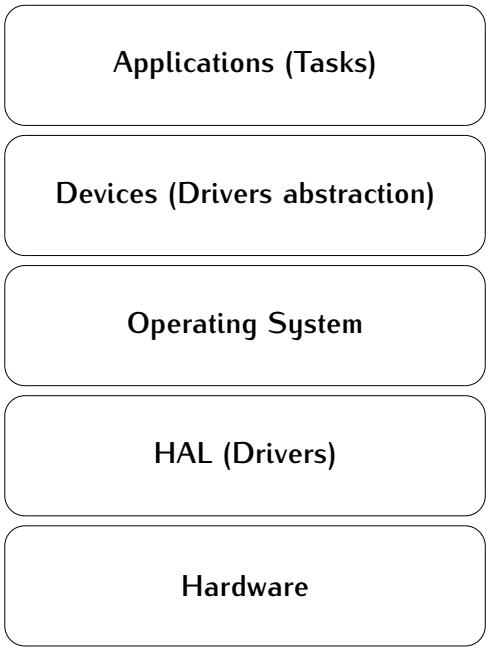


Figure 2.3: System layers.

2.4 Operation

The system operates through the sequential execution of routines (tasks in the context of the operating system) that are scheduled and multiplexed over time. Each routine has a priority and a periodicity, determining the subsequent execution, the set of functionalities currently running, and the memory usage management. Besides this deterministic scheduling system, the routines have communication channels with each other through

the usage of queues, which provides a robust synchronization scheme. In chapter 4 the system operation and the internal nuances are described in detail. Then, this section uses a top-view user perspective to describe the module operation.

2.4.1 Execution Flow

The OBDH 2.0 execution flowchart can be seen on Figure 2.4.
The boot sequence can be seen better on Figure 2.5.
The antenna deploy routine is exemplified with the flowchart on Figure 2.6.
The Telecommand's processing flowchart can be seen on Figure 2.7.
The Transmit/Receive sequence can be seen with the flowchart on Figure 2.8.

2.4.2 Data Flow

The OBDH 2.0 controls most of the CubeSat's data flow, which can be seen on Figure 2.9.

2.4.3 Status LEDs

On the development version of the board, eight LEDs indicate some behaviors of the systems. This set of LEDs can be seen on Figure 2.10.

A description of each of these LEDs are available below:

- **D1 - System LED**: Heartbeat of the system. Blinks at a frequency of 1 Hz when the system is running properly.
- **D2 - Fault LED**: Indicates a critical fault in the system.
- **D3 - UART0 TX**: Blinks when data is being transmitted over the UART0 port.
- **D4 - UART0 RX**: Blinks when data is being received over the UART0 port.
- **D5 - UART1 TX**: Blinks when data is being transmitted over that UART1 port.
- **D6 - UART1 RX**: Blinks when data is being received over the UART1 port.
- **D7 - Antenna VCC**: Indicates that the antenna module board is being power sourced.
- **D8 - OBDH VCC**: Indicates that the OBDH board is being power sourced.

These LEDs are not mounted in the flight version of the module.

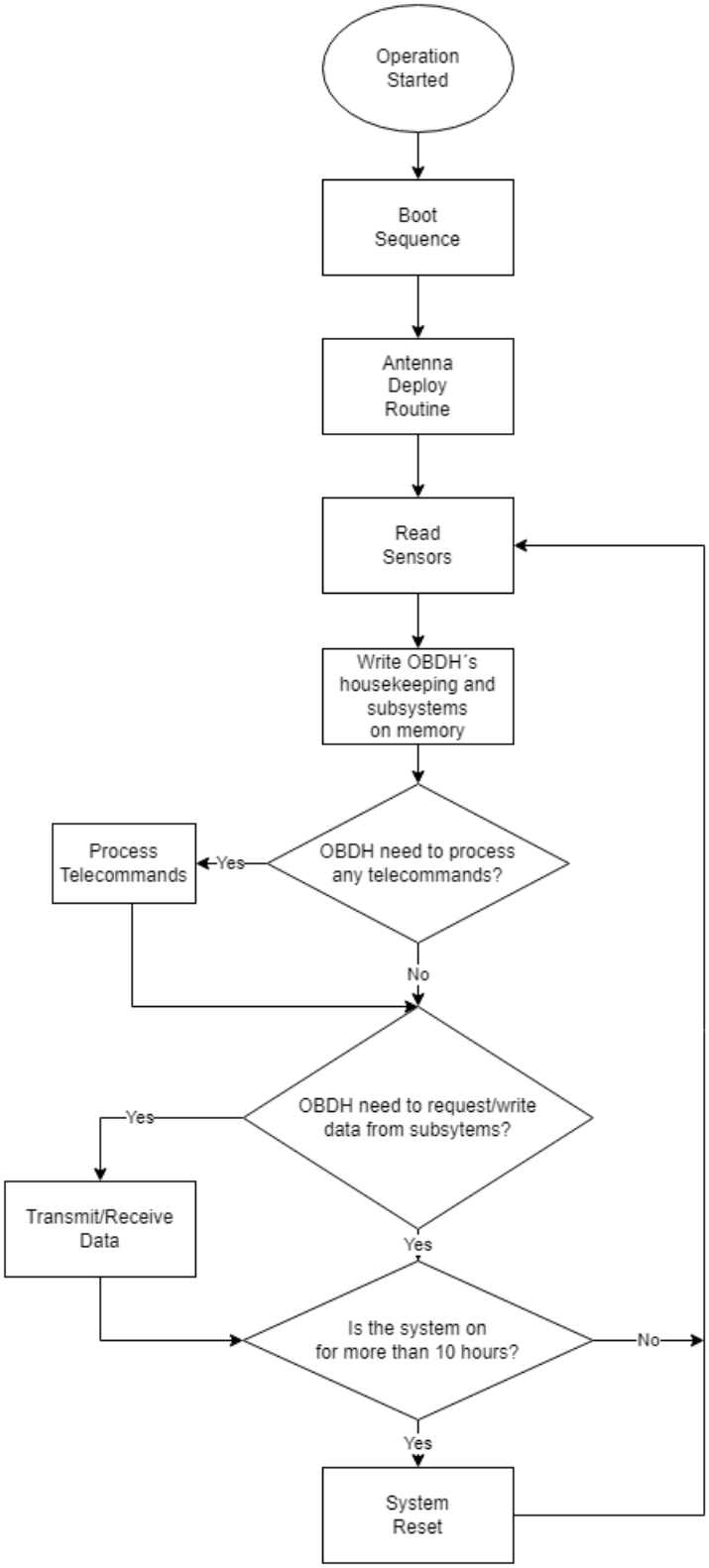


Figure 2.4: OBDH 2.0 operation flowchart

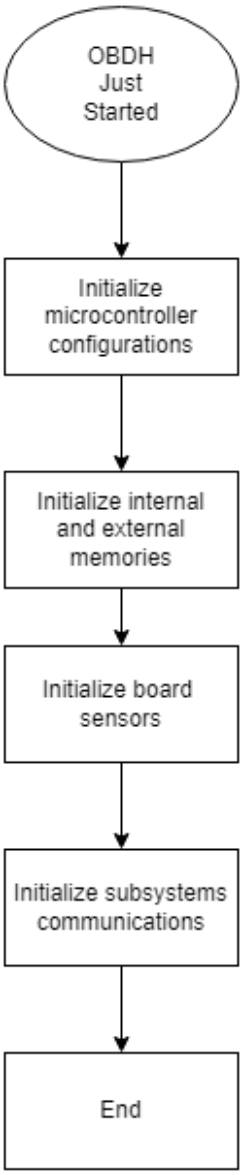


Figure 2.5: OBDH's 2.0 boot sequence.

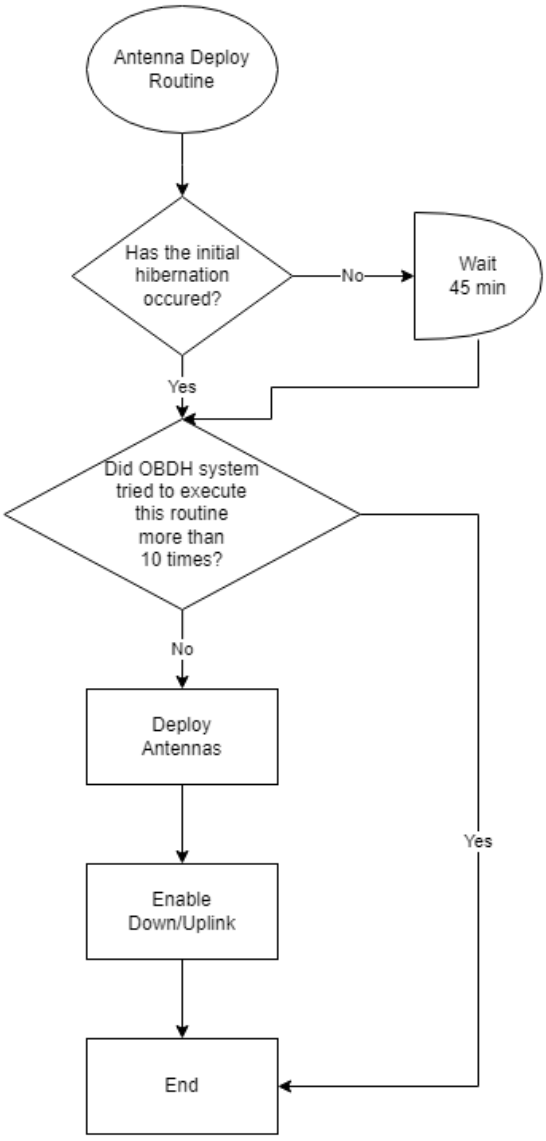


Figure 2.6: Antenna deploy routine

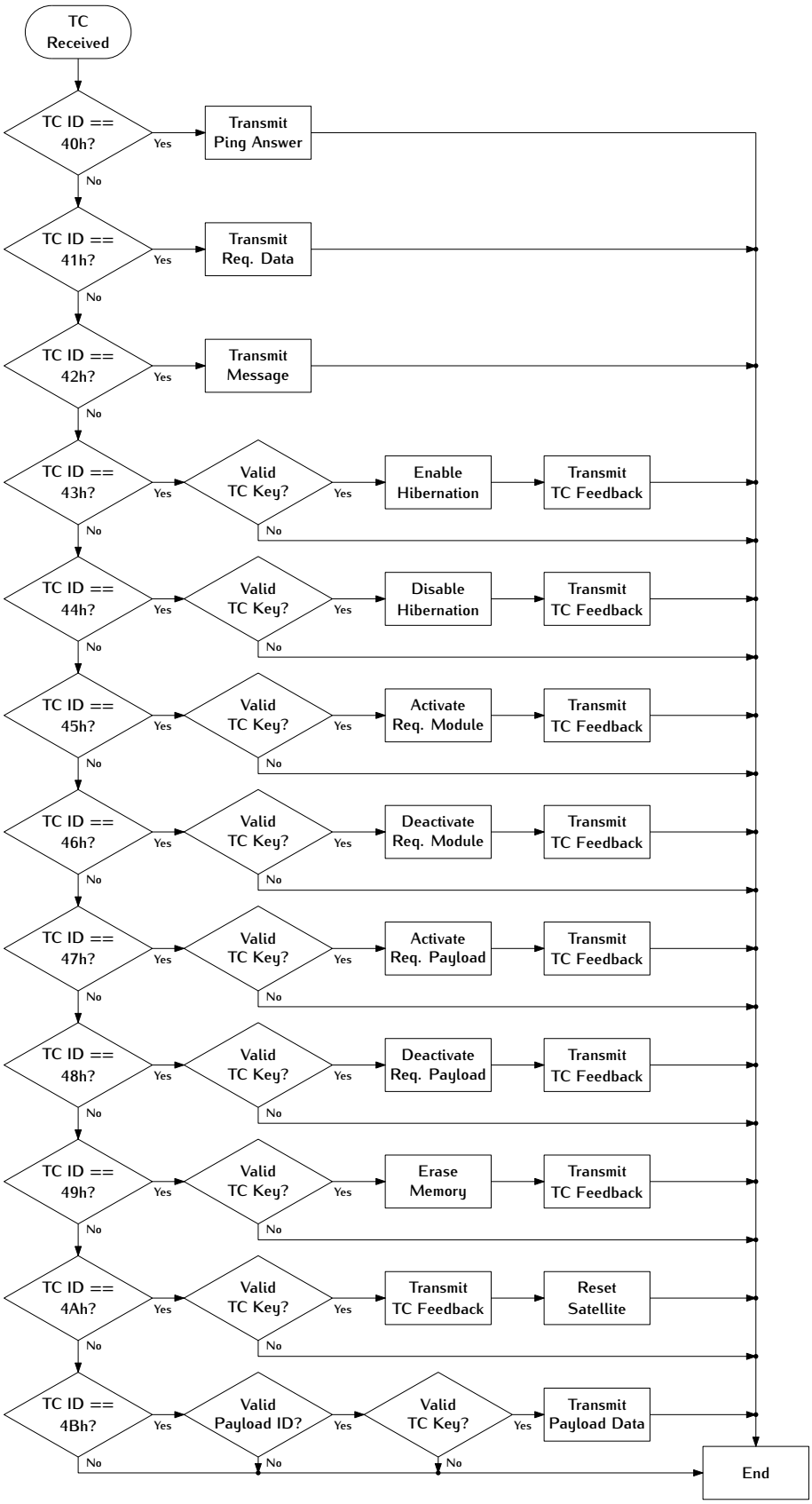


Figure 2.7: Telecommand's processing flowchart

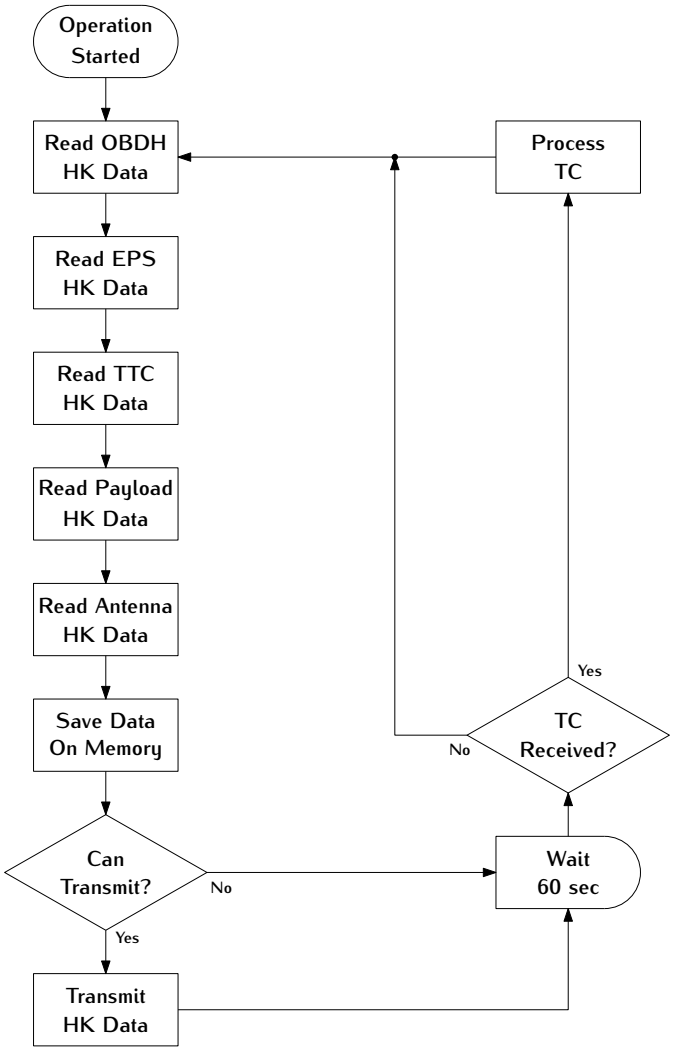


Figure 2.8: Transmit/Receive flowchart

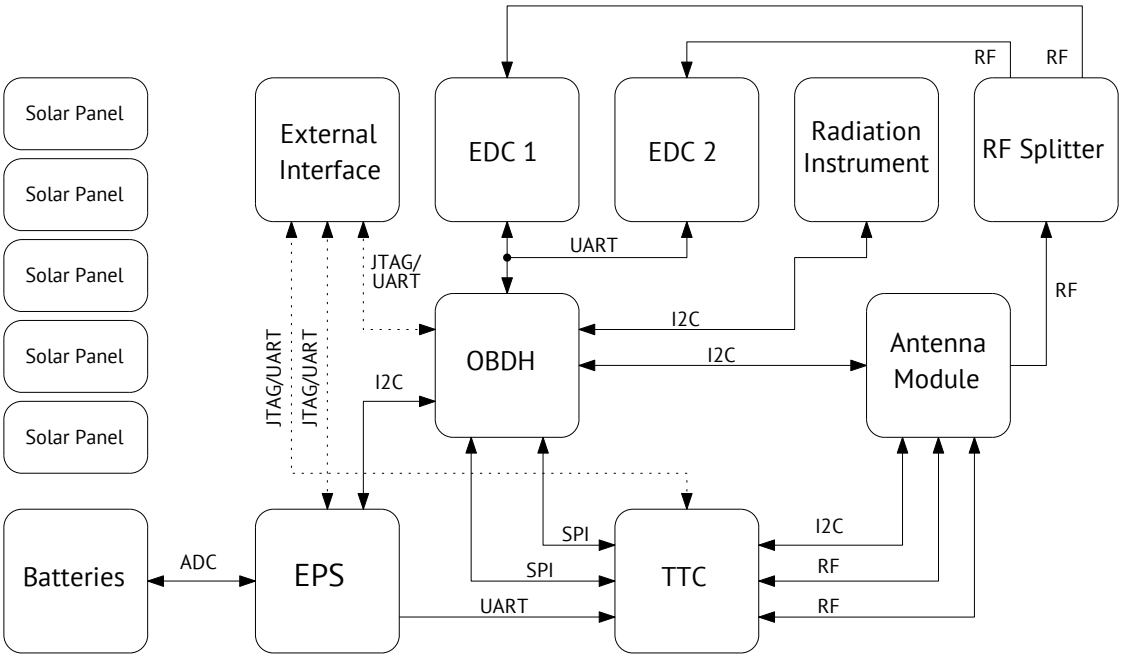


Figure 2.9: Data path diagram.

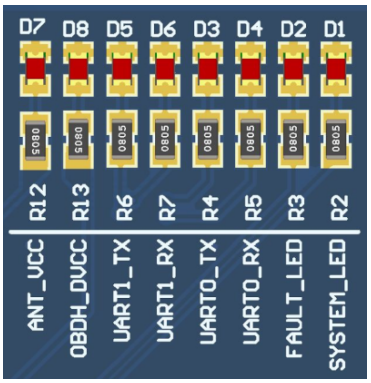


Figure 2.10: Available status LEDs.

CHAPTER 3

Hardware

The OBDH 2.0 architecture focuses on the low-power operation and low-cost production, maintaining performance and proposing different approaches to increase overall reliability. Therefore, the board was developed using these criteria, and the changes from the original design were necessary to improve bottlenecks and achieve the requirements of the further space mission. The Figure 2.2 presents the module architecture from the hardware perspective, including the main PCB components and interfaces: microcontroller, buffers, transceivers, memory, watchdog and voltage monitor, and connectors. The following sections describe the hardware design, interfaces, and standards in detail. The Figures 3.1, 3.2, and 3.3 present 3D-rendered images of the top, bottom, and side views of the board, respectively.

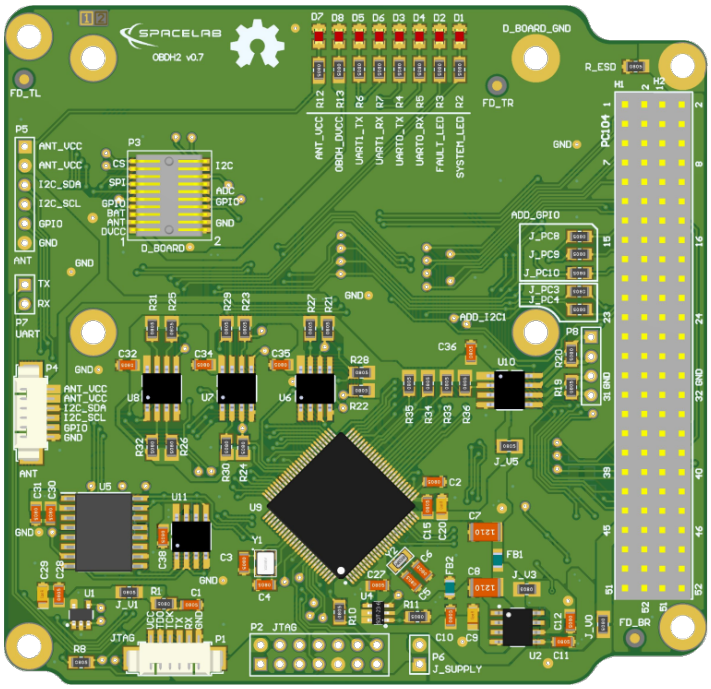


Figure 3.1: Top side of the PCB.

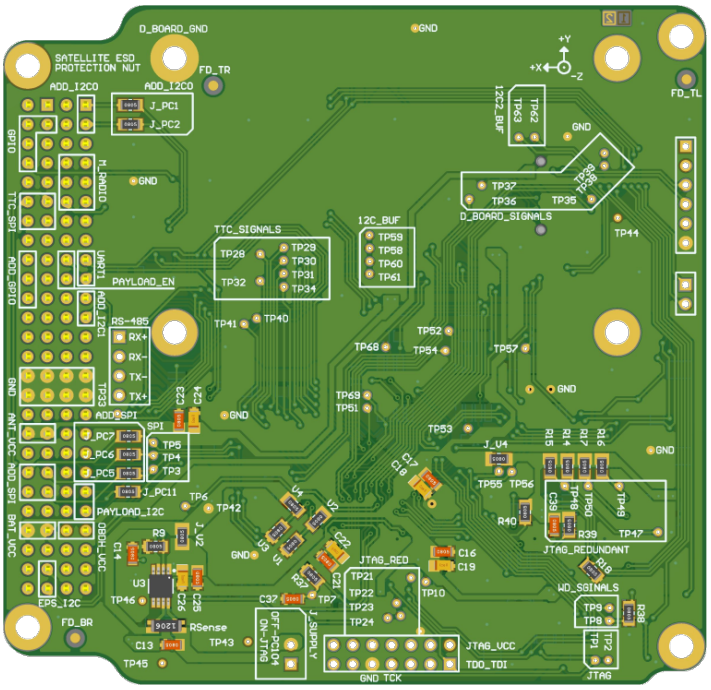


Figure 3.2: Bottom side of the PCB.

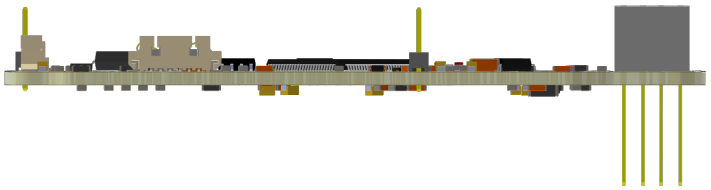


Figure 3.3: Side view of the PCB.

3.1 Interfaces

The Figure 3.4 presents the board interfaces, which consist of communication with other modules, debug access points, and internal peripherals. From the perspective of the microcontroller, there are 6 individual and shared communication buses and the JTAG interface in the following scheme: A0-SPI (shared with Radio, TTC, and external memory chip); A1-UART (shared with redundant payloads); A2-UART (dedicated for debugging); B0-I2C (dedicated for the payload); B1-I2C (dedicated for the EPS); B2-I2C (dedicated for the Antenna module). Currently, "*Payload 1*" and "*Payload 2*" are "*Radiation instrument*" and "*Payload EDC*" respectively.

3.2 External Connectors

The external interfaces are connected to the microcontroller using different connector types: EPS, TTC, Radio, and Payloads through PC-104; Antenna module with 6H header and 6P picoblade connectors; JTAG through 14H header and 6P picoblade connectors; and debug access using a dedicated 2H header and shared with the JTAG connectors. The

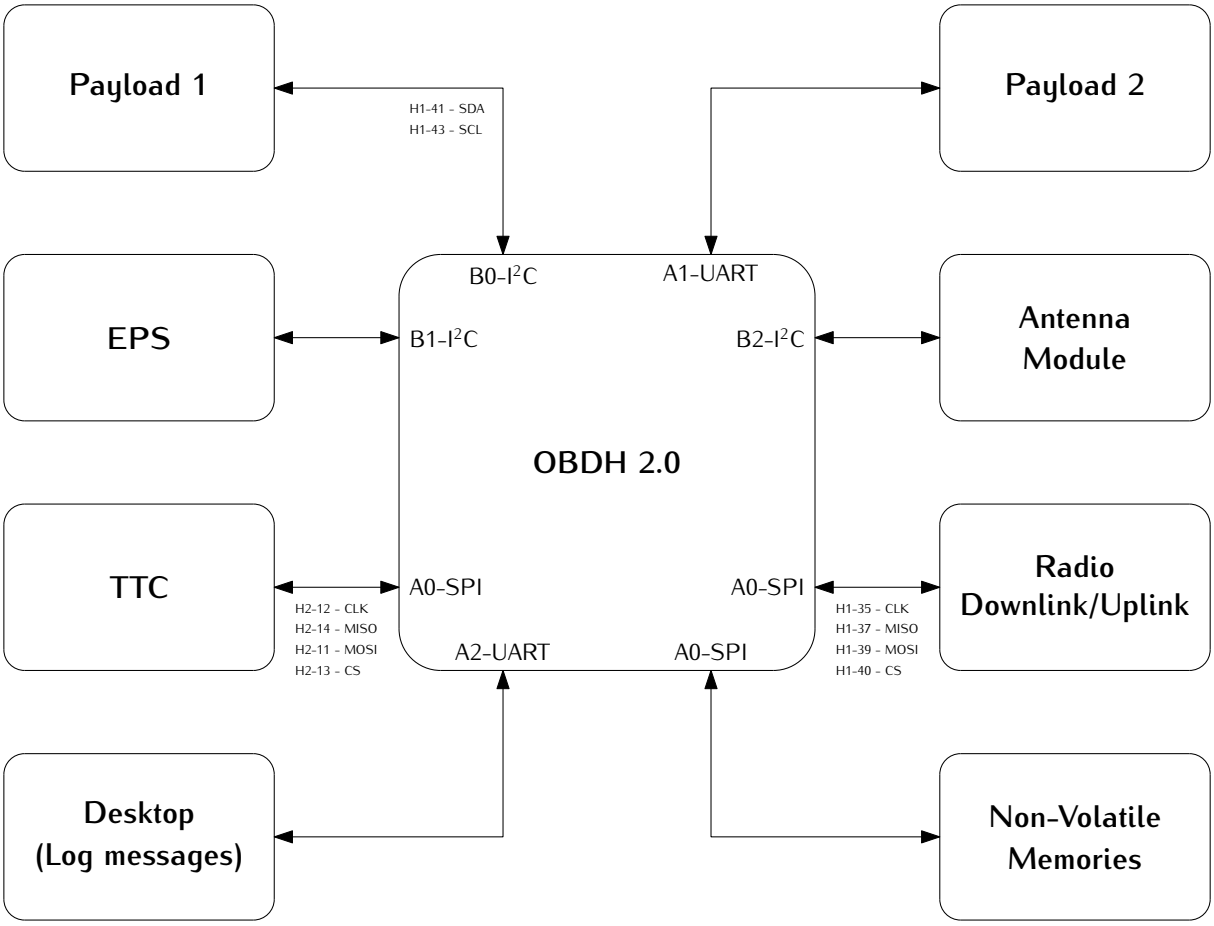


Figure 3.4: Interfaces diagram.

Peripheral	USCI	Protocol	Comm. Protocol
TTC	A0	SPI	Register read/write
Radio (downlink/link)	A0	SPI	Radio config./NGHam
NOR Memory	A0	SPI	-
FRAM Memory	A0	SPI	-
Payload port	A1	UART	-[1]
PC (log messages)	A2	UART	ANSI messages
Payload port	B0	I^2C	-[1]
EPS	B1	I^2C	Register read/write
Antenna Module	B2	I^2C	-

Table 3.1: Boards interfaces.

following topics describe these interfaces and present the pinout of the connectors.

3.2.1 PC-104

The connector PC-104 is a junction of two double-row 28H headers (*SSW-126-04-G-D*). These connectors create a solid 104-pin interconnection across the different satellite

[1]The communication protocol of the payload ports depends of the used payload.

modules. The Figure 3.5 shows the PC-104 interface from the bottom side of the PCB, which allows visualizing the simplified label scheme in the board. Also, the Table 3.2 provides the connector pinout[2] for the pins that are connected to the module.

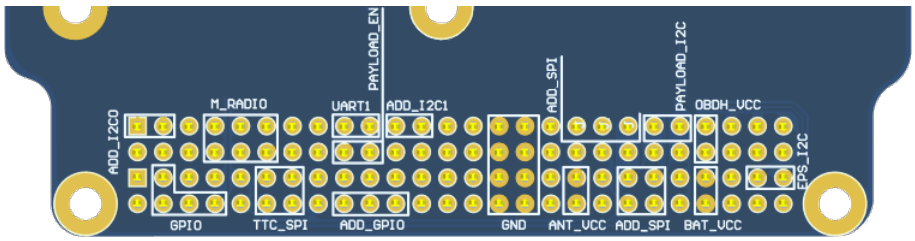


Figure 3.5: Bottom view of PC-104 and simplified labels

Pin [A-B]	H1A	H1B	H2A	H2B
1-2	-	-	-	-
3-4	-	-	GPIO_4	GPIO_5
5-6	-	-	-	-
7-8	GPIO_0	GPIO_1	-	GPIO_6
9-10	GPIO_2	-	-	-
11-12	GPIO_3	GPIO_7	SPI_0_MOSI	SPI_0_CLK
13-14	-	-	SPI_0_CS_1	SPI_0_MISO
15-16	-	-	-	-
17-18	UART_1_RX	GPIO_8	-	-
19-20	UART_1_TX	GPIO_9	-	-
21-22	-	-	-	-
23-24	-	-	-	-
25-26	-	-	-	-
27-28	-	-	-	-
29-30	GND	GND	GND	GND
31-32	GND	GND	GND	GND
33-34	-	-	-	-
35-36	SPI_0_CLK	-	VCC_3V3_ANT	VCC_3V3_ANT
37-38	SPI_0_MISO	-	-	-
39-40	SPI_0_MOSI	SPI_0_CS_0	-	-
41-42	I2C_0_SDA	-	-	-
43-44	I2C_0_SCL	-	-	-
45-46	VCC_3V3	VCC_3V3	VCC_BAT	VCC_BAT
47-48	-	-	-	-
49-50	-	-	I2C_1_SDA	-
51-52	-	-	I2C_1_SCL	-

Table 3.2: PC-104 connector pinout.

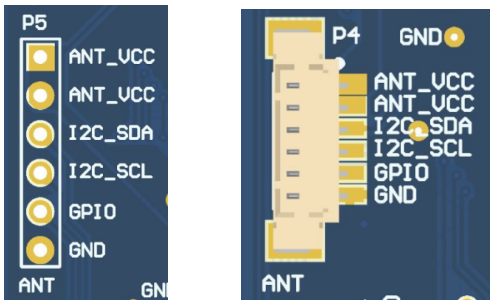
[2]This pinout is simplified since additional interfaces were omitted. Refer to *option sheet* in chapter 5.

3.2.2 Antenna Module

The communication with the Antenna module is performed through external connectors, which are presented in the Figure 3.6. Both connectors have the same connections, but the 3.6(a) (6H header) is used for development, and the 3.6(b) (6P picoblade) as the connector for the flight model. This interface consists of a dedicated I2C, power supply, and GPIO, described in the Table 3.3.

Pin	Row
1	VCC_3V3_ANT
2	VCC_3V3_ANT
3	I2C_SDA
4	I2C_SCL
5	GPIO
6	GND

Table 3.3: Antenna module connectors pinout.



(a) Debug interface of the antenna module.

(b) Main interface of the antenna module.

Figure 3.6: Antenna module conectors.

3.2.3 Programmer and Debug

The interface with the microcontroller programmer is performed through external connectors, which are presented in the Figure 3.7. Both connectors have the same JTAG and UART interfaces. However, the 14H header is used during development, and the 6P picoblade (provides a more compact and reliable attachment) as the connector for the flight model, which is described in the Table 3.4 and Table 3.5, respectively. This interface consists of a dedicated debug UART, a JTAG, and an external power supply. The debug UART connection has another access point in a dedicated 2H header (P7), as shown in Figure 3.8. Also, to use this external supply, it is necessary to connect both pins of a 2H header jumper (P6).

Figure 3.7: Programmer (P1 and P2) and jumper (P6) connectors.

Pin [A-B]	Row A	Row B
1-2	TDO_TDI	VCC_3V3
3-4	-	-
5-6	-	-
7-8	TCK	-
9-10	GND	-
11-12	-	UART_TX
13-14	-	UART_RX

Table 3.4: Programmer header connector pinout.

Pin	Row
1	VCC_3V3
2	TDO_TDI
3	TCK
4	UART_TX
5	UART_RX
6	GND

Table 3.5: Programmer picoblade connector pinout.

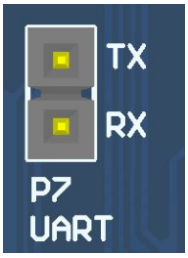


Figure 3.8: Dedicated UART debug connectors (P7).

3.2.4 Daughterboard

The daughterboard interface uses the Samtec FSI-110-D connector [4], which can be seen in the Figure 3.9. This connector has metal contacts in the format of flexible arcs and four polymer guide pins (a pair for the top and bottom). When the daughterboard is attached, there is some pressure on the metal contacts that bend and create a meaningful

pin connection to the daughterboard copper pads[3]. A picture of this connector on the PCB can be seen in Figure 3.10.

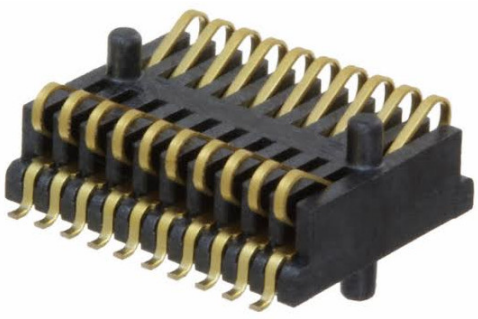


Figure 3.9: Samtec FSI-110-03-G-D-AD connector.

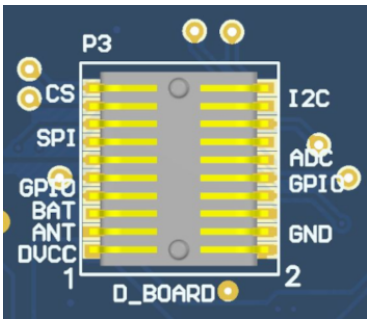


Figure 3.10: Daughterboard connector (P3).

The pinout of the daughterboard interface is available in the Table 3.6. There are different power supply lines (OBDH, Antenna, and battery), communication buses (I2C and SPI), GPIO, and ADC interfaces available. Besides the GPIO and ADC pins, the other interfaces are shared with other modules and peripherals.

Guidelines

The recommended shape and size of the daughterboard can be seen in the Figure 3.11. Besides that, there are mandatory and suggested elements placement: four M3 holes for mechanical attachment, required; contact connector pads (in light gray on the bottom layer), required; two debug headers on the left and bottom sides, suggested; and a general purpose flight model picoblade suggested.

3.3 Microcontroller

The OBDH 2.0 uses a low-power and low-cost microcontroller family from Texas Instruments; the MSP430F6659 [5]. This device provides sufficient performance for low and medium-complexity software and algorithms, allowing the module to execute the required

[3]These daughterboard pads are similar to the ones used as a footprint in the OBDH, despite a slightly bigger size.

Pin [A-B]	Row A	Row B
1-2	VCC_3V3	GND
3-4	VCC_3V3_ANT	GND
5-6	VCC_BAT	GND
7-8	GPIO_0	GPIO_1
9-10	GPIO_2	GPIO_3
11-12	SPI_0_CLK	ADC_0
13-14	SPI_0_MISO	ADC_1
15-16	SPI_0_MOSI	ADC_2
17-18	SPI_0_CS_0	I2C_2_SDA
19-20	SPI_0_CS_1	I2C_2_SCL

Table 3.6: Daughterboard connector pinout.

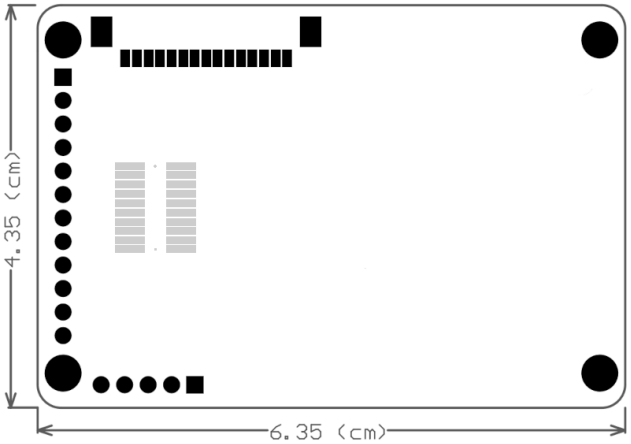


Figure 3.11: Recommended shape and size of the daughterboard.

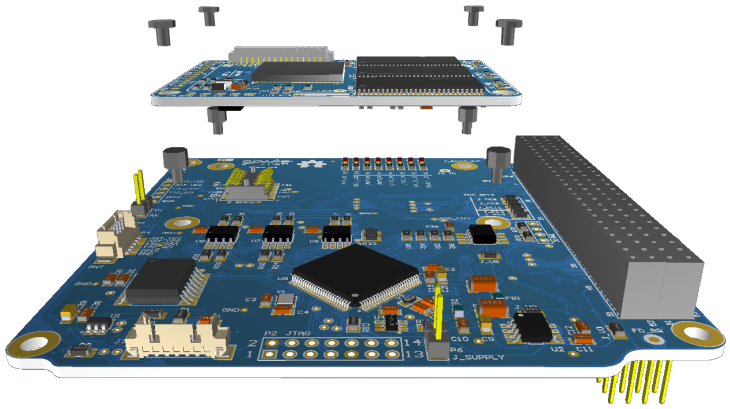


Figure 3.12: Illustrative daughterboard integration.

tasks. The Table 3.7 presents a summary of the main available features and Figure 3.13 shows the internal subsystems, descriptions, and peripherals. The microcontroller interfaces, configurations, and auxiliary components are described in the following topics.

Flash	SRAM	Timers	USCI	ADC	DAC	GPIO
512KB	64KB	2	6 (SPI / I2C / UART)	12	2	74

Table 3.7: Microcontroller features summary.

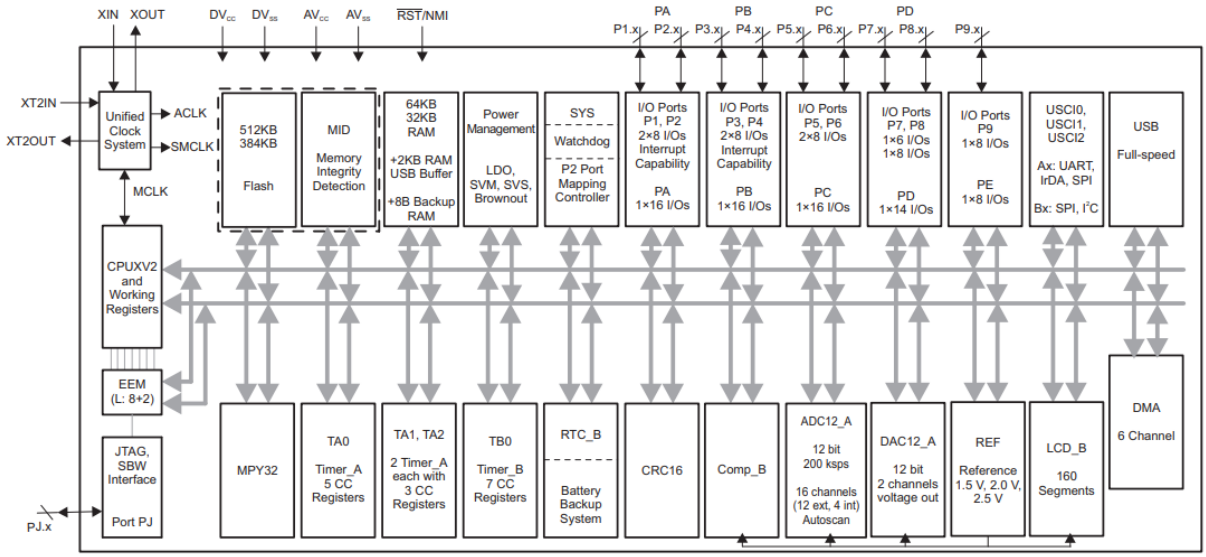


Figure 3.13: Microcontroller internal diagram.

3.3.1 Interfaces Configuration

The microcontroller has 6 Universal Serial Communication Interfaces (USCI) that can be configured to operate with different protocols and parameters. These interfaces are connected to different modules and peripherals, as presented in the Figure 3.4. The Table 3.8 describes each interface configuration.

Interface	Protocol (Index)	Mode	Word Length	Data Rate	Configuration
USCI_A0	SPI	Master	8 bits	1 Mbps	Phase: High Polarity: Low
USCI_A1	UART1	-	8 bits	115200 bps	Stop bits: 1 Parity: None
USCI_A2	UART0	-	8 bits	115200 bps	Stop bits: 1 Parity: None
USCI_B0	I2C0	Master	8 bits	100 kbps	Adr. len: 7 bits
USCI_B1	I2C1	Master	8 bits	100 kbps	Adr. len: 7 bits
USCI_B2	I2C2	Master	8 bits	100 kbps	Adr. len: 7 bits

Table 3.8: USCI configuration.

3.3.2 Clocks Configuration

Besides the internal clock sources, the microcontroller has two dedicated clock inputs for external crystals: the main clock and the auxiliary. A 32 *MHz* crystal and a 32.769 *kHz*

are connected to these inputs. The first source is used for generating the Master Clock (MCLK) and the Subsystem Master Clock (SMCLK), which are used by the CPU and the internal peripheral modules. The second source is used for generating the Auxiliary Clock (ACLK) that handles the low-power modes and might be used for peripherals.

3.3.3 Pinout

An illustration of the microcontroller pinout positions can be seen in the Figure 3.14. The Table 3.9 presents the OBDH 2.0 microcontroller pins assignment.

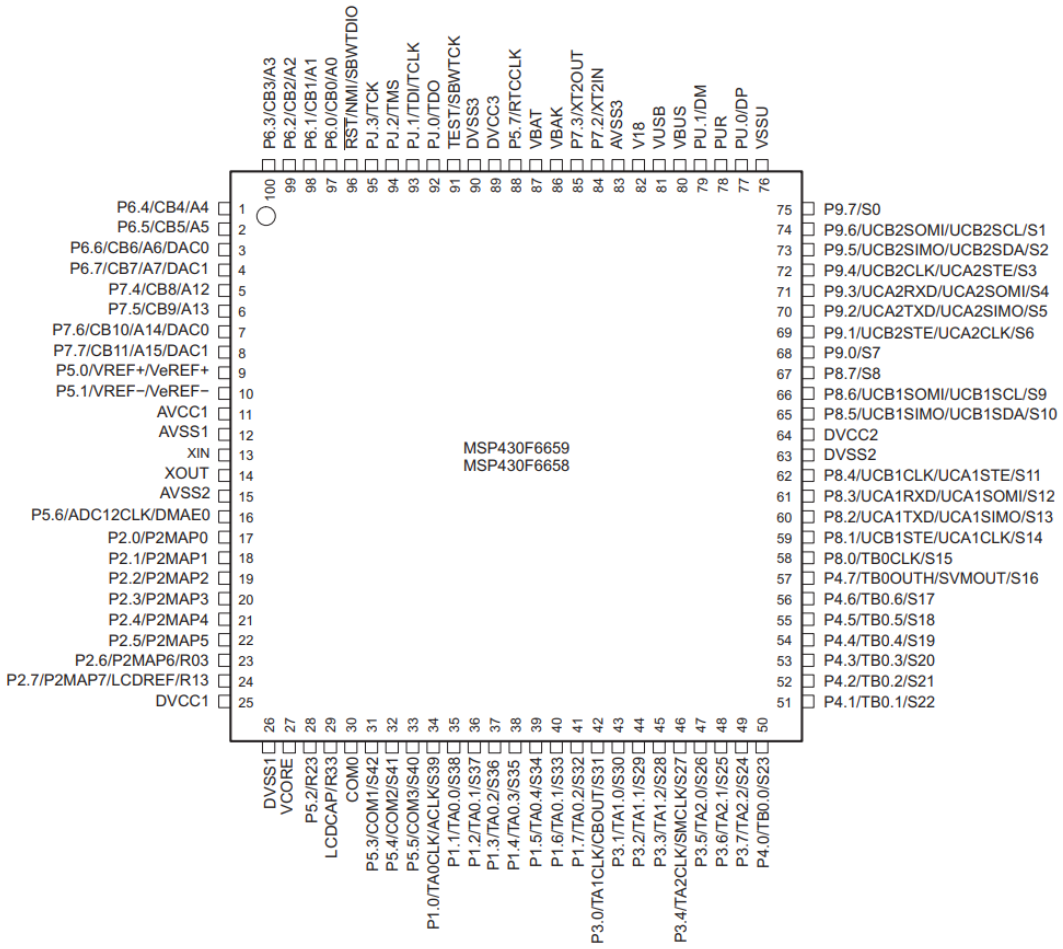


Figure 3.14: Microcontroller pinout positions.

Pin Code	Pin Number	Signal
P1.0	34	MAIN_RADIO_ENABLE
P1.1	35	MAIN_RADIO_GPIO0
P1.2	36	MAIN_RADIO_GPIO1
P1.3	37	MAIN_RADIO_GPIO2
P1.4	38	MAIN_RADIO_RESET
P1.5	39	MAIN_RADIO_SPI_CS
P1.6	40	TTC_MCU_SPI_CS

P1.7	41	-
P2.0	17	SPI_CLK
P2.1	18	I2C0_SDA
P2.2	19	I2C0_SCL
P2.3	20	-
P2.4	21	SPI_MOSI
P2.5	22	SPI_MISO
P2.6	23	VERSION_BIT0
P2.7	24	VERSION_BIT1
P3.0	42	I2C0_EN
P3.1	43	I2C1_EN
P3.2	44	I2C2_EN
P3.3	45	I2C0_READY
P3.4	46	I2C1_READY
P3.5	47	I2C2_READY
P3.6	48	PC104_GPIO0
P3.7	49	PC104_GPIO1
P4.0	50	PC104_GPIO2
P4.1	51	PC104_GPIO3
P4.2	52	MEM_HOLD
P4.3	53	MEM_RESET
P4.4	54	MEM_SPI_CS
P4.5	55	PC104_GPIO4
P4.6	56	PC104_GPIO5
P4.7	57	PC104_GPIO6
P5.0	9	VREF
P5.1	10	AGND
P5.2	28	SYSTEM_FAULT_LED
P5.3	31	SYSTEM_LED
P5.4	32	PAYLOAD_0_ENABLE
P5.5	33	PAYLOAD_1_ENABLE
P5.6	16	-
P5.7	88	-
P6.0	97	D_BOARD_ADC0
P6.1	98	D_BOARD_ADC1
P6.2	99	D_BOARD_ADC2
P6.3	100	OBDH_CURRENT_ADC
P6.4	1	OBDH_VOLTAGE_ADC
P6.5	2	D_BOARD_SPI_CS0
P6.6	3	D_BOARD_SPI_CS1
P6.7	4	-
P7.0	-	-
P7.1	-	-
P7.2	84	XT2_N
P7.3	85	XT2_P
P7.4	5	D_BOARD_GPIO0

P7.5	6	D_BOARD_GPIO1
P7.6	7	D_BOARD_GPIO2
P7.7	8	D_BOARD_GPIO3
P8.0	58	-
P8.1	59	-
P8.2	60	UART1_TX
P8.3	61	UART1_RX
P8.4	62	-
P8.5	65	I2C1_SDA
P8.6	66	I2C1_SCL
P8.7	67	ANTENNA_GPIO
P9.0	68	FRAM_WP
P9.1	69	FRAM_SPI_CS
P9.2	70	UART0_TX
P9.3	71	UART0_RX
P9.4	72	WDI_EXT
P9.5	73	I2C2_SDA
P9.6	74	I2C2_SCL
P9.7	75	MR_WDOG
PJ.0	92	TP21
PJ.1	93	TP22
PJ.2	94	TP23
PJ.3	95	TP24
-	13	XT1IN
-	14	XT1OUT
-	96	JTAG_TDO_TDI
-	91	JTAG_TCK

Table 3.9: Microcontroller pinout and assignments.

3.4 External Watchdog

In addition to the internal watchdog timer of the microcontroller, to ensure a system reset in case of a software freeze, an external watchdog circuit is being used. For that, the TPS3823 IC from Texas Instruments [6] was chosen. This IC is a voltage monitor with a watchdog timer feature. This circuit can be seen in the Figure 3.15.

This circuit works this way: if the WDI pin remains high or low longer than the timeout period, then reset is triggered. The timer clears when reset is asserted or when WDI sees a rising or falling edge.

The watchdog timer task clears the TPS3823 timer by toggling the WDI pin every 100 *ms*. If the WDI pin state stays unmodified for more than 1600 *ms*, the reset pin is cleared, and the microcontroller is reset.

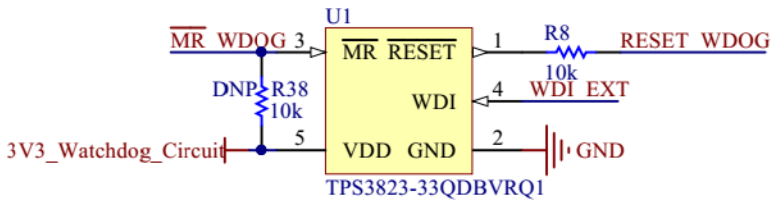


Figure 3.15: External watchdog timer circuit.

3.5 Non-Volatile Memories

There are two non-volatile memories available on the module: one flash NOR memory and one FRAM memory.

3.5.1 Flash NOR

The flash NOR non-volatile memory model is the Micron MT25QL01GBBB, which is composed of a NOR flash architecture with 1 Gb of capacity (or 128 MB) and features extended SPI configurations. As seen in Figure 3.4, an SPI bus is used to communicate with this peripheral, using the Table 3.8 configurations. Also, some control pins are connected to microcontroller GPIOs: HOLD#, RESET#, and W#.

When RESET# is driven LOW, the device is reset, and the outputs are tri-stated. The HOLD# signal pauses serial communications without deselecting or resetting the device; outputs are tri-stated, and inputs are ignored. The W# signal handles as write protection, freezes the status register, turning its non-volatile bits read-only and preventing the write operation from being executed.

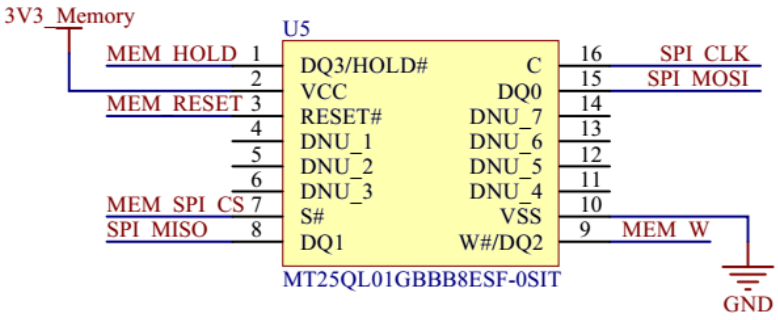


Figure 3.16: External memory circuit.

3.5.2 FRAM

The EXCELON™ Auto CY15X102QN is an automotive grade, 2Mb non-volatile memory employing an advanced ferroelectric process. A ferroelectric random access memory or F-RAM is non-volatile and performs reads and writes similar to RAM. It provides reliable data retention for 121 years. The schematics of the memory can be seen in Figure 3.17, an SPI bus is used to communicate with this peripheral.

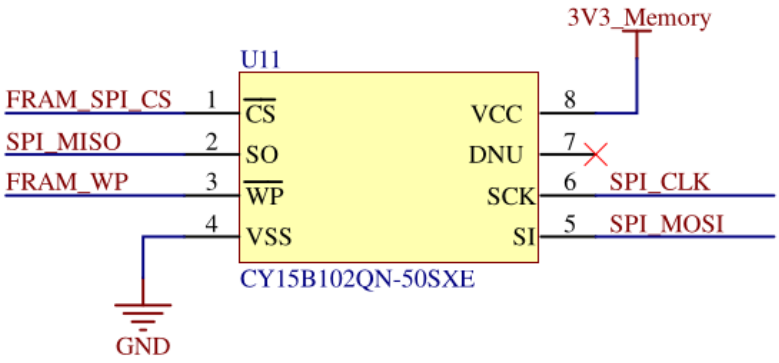


Figure 3.17: FRAM memory circuit.

3.6 I2C Buffers

The microcontroller I2C interfaces have dedicated IC buffers, which improve the signal quality throughout the various connectors and offer reliability enhancements since it protects the bus in case of failures. This measure was adopted in all the satellite modules due to previous failures in I2C buses. Using this scheme, the modules connected through this protocol might have shared connections without losing performance or reliability.

The buffer selected for this function is the Texas Instruments TCA4311 device. Besides the I2C inputs and outputs, it features control and status signals that are connected to GPIOs in the microcontroller: an enable and an operation-ready status. Also, both inputs and outputs in these I2C lines have external pull-up resistors.

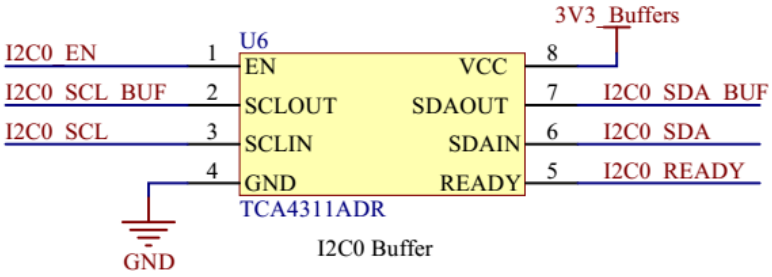


Figure 3.18: I2C buffer circuit.

3.7 RS-485 Transceiver

The module features an RS-485 interface connected to a 4H header (P8). This interface uses a transceiver (THVD1451) to convert the incoming RS-485 signals to UART and vice-versa. The outputs are 120 Ω differential pairs that have termination resistors before connecting to the header pins.

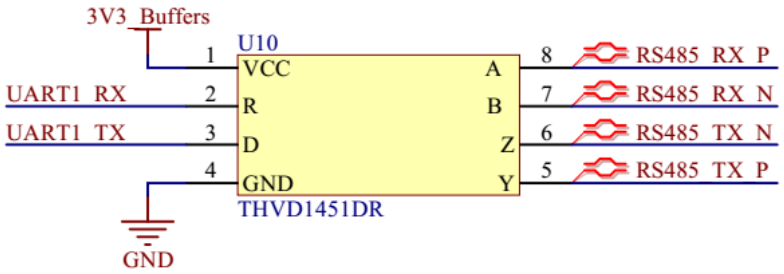


Figure 3.19: RS-485 transceiver circuit.

3.8 Voltage and Current Sensors

To monitor the board's overall current and voltage, the module has a current sensor using a Maxim Integrated IC (MAX9934) and a buffered voltage divider circuit with a Texas Instruments IC (TLV341A). These circuits have direct analog outputs that are connected to ADC inputs. The microcontroller's internal ADC peripheral has a dedicated input for a voltage reference, which is connected to the REF5030A IC. This device generates a precise 3 V output that enhances the measures and conversions performed by the microcontroller.

CHAPTER 4

Firmware

4.1 Product tree

The product tree of the firmware part of the OBDH 2.0 module is available in Figure 4.1.

4.2 Dependencies

The firmware depends on external libraries to access the embedded hardware or to communicate with other modules. A list of these libraries and the used version is available in Table 4.1.

Library	Version
MSP430 DriverLib	v2.91.11.01
FreeRTOS	v10.2.1

Table 4.1: External libraries and dependencies of the firmware.

4.3 Tasks

A list of the firmware tasks can be seen in the Table 4.2.
All these tasks are better described below.

4.3.1 Antenna deployment

This task deploys the Antenna module at the start of the mission.

4.3.2 Antenna reading

Reads the Antenna module status.

4.3.3 Data log

This task saves the housekeeping data of the satellite in flash memory every 10 minutes.

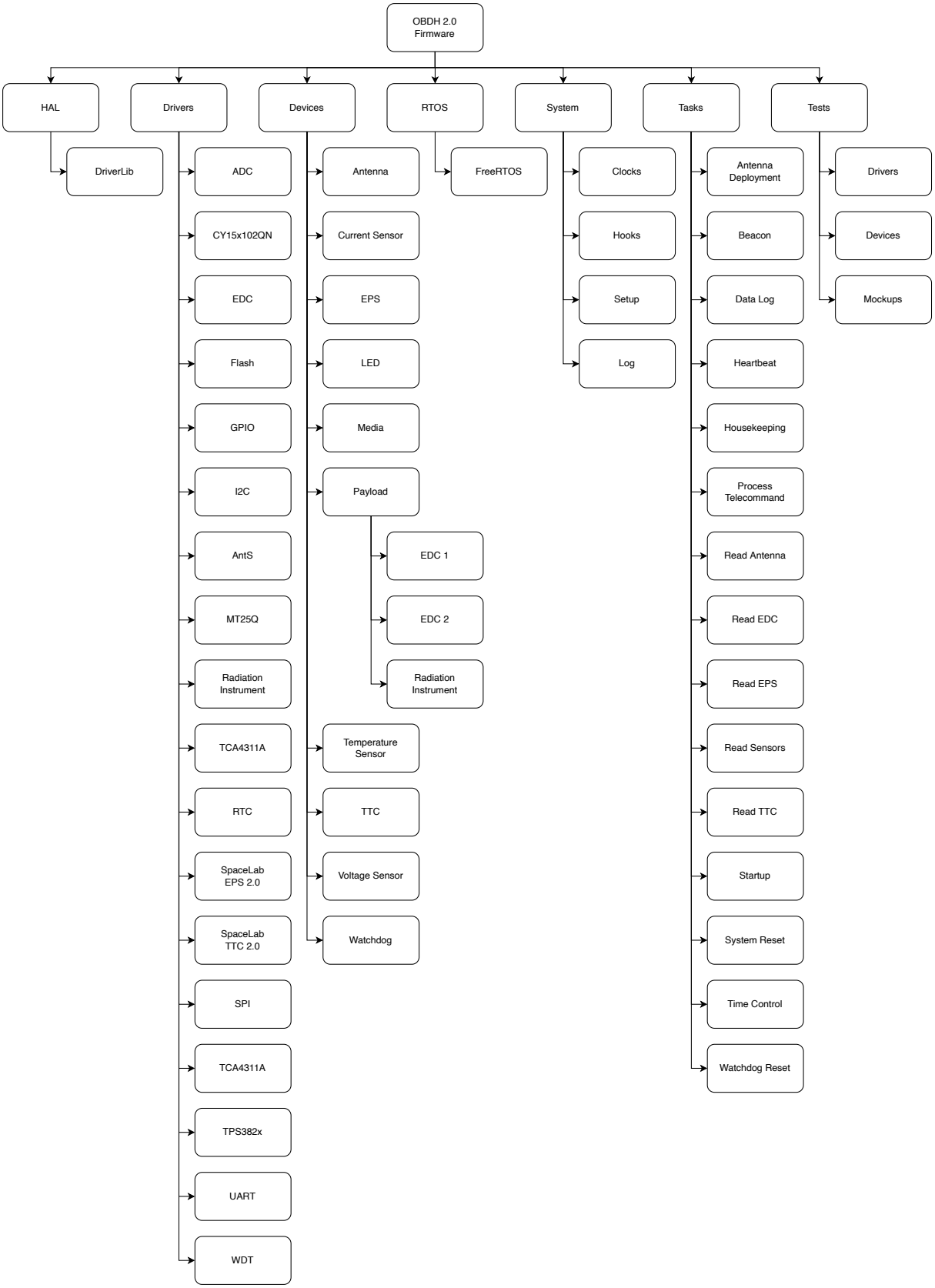


Figure 4.1: Product tree of the firmware of the OBDH 2.0 module.

Name	Priority	Initial delay [ms]	Period [ms]	Stack [bytes]
Antenna deployment	Highest	0	Aperiodic	150
Antenna reading	Medium	2000	60000	150
Data log	Medium	2000	600000	225
General Telemetry	High	10000	60000	225
EDC reading	Medium	2000	60000	300
EPS reading	Medium	2000	60000	384
Heartbeat	Lowest	2000	500	160
Housekeeping	Medium	2000	60000	225
Mission Manager	High	0	Aperiodic	512
Payload X reading	Medium	2000	60000	300
Position Determination	Low	1000	60000	1024
Read sensors	Medium	2000	60000	140
Startup (boot)	Highest	0	Aperiodic	350
System reset	Medium	0	36000000	128
Telecommand processing	High	16500	1000	1024
Time control	Medium	1000	1000	128
TTC reading	Medium	7500	60000	384
Watchdog reset	Lowest	0	100	150

Table 4.2: Firmware tasks.

4.3.4 General Telemetry

The General Telemetry task transmits a data package containing the satellite's basic telemetry data every 60 seconds.

4.3.5 EDC reading

This task reads all the EDC packages and data.

4.3.6 EPS reading

This task reads all the EPS data and status.

4.3.7 Heartbeat

The heartbeat task keeps blinking a LED ("*System LED*" in Figure 2.10) at a rate of 1 Hz during the execution of the system. Its purpose is to give visual feedback on the execution of the scheduler. This task does not have a specific purpose on the flight version of the module (the flight version of the PCB does not have LEDs).

4.3.8 Housekeeping

This task saves the OBDH data to FRAM. It also checks for hibernation mode duration.

4.3.9 Mission Manager

This task controls all mission specific behavior, specially payload control.

4.3.10 Payload X reading

This task reads all Payload X data and status.

4.3.11 Position Determination

This task determines the satellite position based on TLE lines provided via telecommands.

4.3.12 Read sensors

This task reads the internal sensors of the OBDH every 60 seconds.

4.3.13 Startup (boot)

This task is the first executed task when the system starts. All devices, libraries, and data structures are initialized in this task. When the execution is done, the remaining tasks of the system are allowed to execute.

4.3.14 System reset

This task resets the microcontroller by software every 10 hours. This can be useful to clean up possible wrong values in variables, repeat the antenna deployment routine (limited to n times), clean up the RAM, etc.

4.3.15 Telecommand processing

This task processes all the telecommands; it needs to have a really short period for a more responsive operation.

4.3.16 Time control

This task is responsible for the time management of the system. At every second, it increments the system time (epoch). Also, it saves the current system time in the non-volatile memory every minute.

4.3.17 TTC reading

This task reads all the TTC data and status.

4.3.18 Watchdog reset

This task resets the internal and external watchdog timer every 100 ms. The internal watchdog has a maximum count time of 500 ms, and the external watchdog has a maximum of 1600 ms (see chapter 3 for more information about the watchdog timers).

To prevent the system to not reset during an anomaly on some task (like an execution time longer than planned), this task has the lowest possible priority: 0.

4.4 Variables and Parameters

The internal variables and parameters of the OBDH firmware can be seen in Table 4.3.

ID	Name/Description	Type
0	Time counter in milliseconds	uint32
1	Temperature of the μC in Kelvin	uint16
2	Input current in mA	uint16
3	Input voltage in mV	uint16
4	Last reset cause: - 0x00 = No interrupt pending - 0x02 = Brownout (BOR) - 0x04 = RST/NMI (BOR) - 0x06 = PMMSWBOR (BOR) - 0x08 = Wakeup from LPMx.5 (BOR) - 0x0A = Security violation (BOR) - 0x0C = SVSL (POR) - 0x0E = SVSH (POR) - 0x10 = SVML_OVP (POR) - 0x12 = SVMH_OVP (POR) - 0x14 = PMMSWPOR (POR) - 0x16 = WDT time out (PUC) - 0x18 = WDT password violation (PUC) - 0x1A = Flash password violation (PUC) - 0x1C = Reserved - 0x1E = PERF peripheral/configuration area fetch (PUC) - 0x20 = PMM password violation (PUC) - 0x22 to 0x3E = Reserved	uint8
5	Reset counter	uint16
6	Last valid telecommand (uplink packet ID)	uint8
7	Temperature of the radio in Kelvin	uint16
8	RSSI of the last valid telecommand	uint16
9	Temperature of the antenna in Kelvin	uint16
	Antenna status bits: - Bit 15: The antenna 1 is deployed (0) or not (1) - Bit 14: Cause of the latest activation stop for antenna 1 - Bit 13: The antenna 1 deployment is active (1) or not (0) - Bit 11: The antenna 2 is deployed (0) or not (1) - Bit 10: Cause of the latest activation stop for antenna 2	

	- Bit 9: The antenna 2 deployment is active (1) or not (0) - Bit 8: The antenna is ignoring the deployment switches (1) or not (0) - Bit 7: The antenna 3 is deployed (0) or not (1) - Bit 6: Cause of the latest activation stop for antenna 3 - Bit 5: The antenna 3 deployment is active (1) or not (0) - Bit 4: The antenna system independent burn is active (1) or not (0) - Bit 3: The antenna 4 is deployed (0) or not (1) - Bit 2: Cause of the latest activation stop for antenna 4 - Bit 1: The antenna 4 deployment is active (1) or not (0) - Bit 0: The antenna system is armed (1) or not (0)	
11	Hardware version	uint8
12	Firmware version (ex.: "v1.2.3" = 0x00010203)	uint32
13	Mode ("Normal" = 0, "Hibernation" = 1)	uint8
14	Timestamp of the last mode change	uint32
15	Mode duration in sec. (valid only in hibernation mode)	uint32
16	Initial hibernation executed	boolean
17	Initial hibernation time counter (minutes)	uint8
18	Antenna deployment executed	boolean
19	Antenna deployment counter	uint8

Table 4.3: Variables and parameters of the OBDH 2.0.

4.5 Telemetry

All telemetry data available to read from OBDH is expected to be serialized in big endian ordering, so a data field named "param" with the type uint16, is serialized as the first byte being the 8 most significant bits of param and the second byte is the 8 least significant bits of it. This happens to all data types with length bigger than 1 byte.

The following subsections will present the available telemetry information for each subsystem, keep in mind that this is only the data field of the complete downlink packet, there is still the IDs, callsigns, etc. To see a complete representation of the packet look at the Table A.1.

4.5.1 EDC Information

Telemetry packages from EDC can be seen in Table 4.4. This data can be requested through the "Get Payload Data" and "Data Request" telecommands, for more information about them see section 4.6. PTT packets from the Sistema Brasileiro de Coleta de Dados (SBCD) are seen in Table 4.9.

4.5.2 OBDH Information

Telemetry packages from OBDH can be seen in Table 4.5. This data can be requested through the "Data Request" telecommands, for more information about it, see section 4.6.

Position	Content	Data type	Len. [bytes]
0	System time on which EDC data was last read	uint32	4
	HK Info		
4	Current time since J2000 epoch	uint32	4
8	Elapsed time since last reset	uint32	4
12	Digital circuitry current supply in mA	uint16	2
12	Analog circuitry supply in mA	uint16	2
16	System voltage supply in mV	uint16	2
18	EDC board temperature	int8	1
19	RF front end LO...	uint8	1
20	RMS level at front-end output	uint16	2
22	Generated PTT packages since	uint32	4
26	Maximum number of active PTT decoder channels	uint8	1
27	Number of double bit errors detected	uint8	1
	System State		
28	Current time since J2000 epoch	uint16	4
32	Number of PTT Package available for reading	uint8	1
33	PTT decoder task status	uint8	1
34	ADC sampler state	uint8	1
35	Payload ID (EDC_1 or EDC_2)	uint8	1
Total	-	-	36

Table 4.4: EDC information packet.

4.5.3 EPS Information

Telemetry packages from EPS can be seen in Table 4.6. This data can be requested through the "Data Request" telecommands, for more information about it, see section 4.6.

4.5.4 TTC Information

Telemetry packages from TTC can be seen in Table 4.7. This data can be requested through the "Data Request" telecommands, for more information about it, see section 4.6.

4.5.5 Antenna Information

Telemetry packages from Antenna can be seen in Table 4.8. This data can be requested through the "Data Request" telecommands, for more information about it, see section 4.6.

4.5.6 SBCD Packets

Telemetry packages from the SBCD can be seen in Table 4.9. This data can be requested through the "Data Request" telecommands, for more information about it, see section 4.6.

Position	Content	Data type	Len. [bytes]
0	System time	uint32	4
4	μC temperature in Kelvin	uint16	2
6	μC current supply in mA	uint16	2
8	μC voltage supply in mV	uint16	2
10	Reset counter	uint16	2
12	Last reset cause ID	uint8	1
13	Last valid TC ID	uint8	1
14	Antenna deployment attempts counter	uint8	1
15	Initial hibernation time count (minutes)	uint8	1
16	Hardware version	uint8	1
17	Firmware version	uint32	4
21	Operation mode	uint8	1
22	System time of last mode change	uint32	4
26	Current operation mode duration	uint32	4
30	Initial hibernation executed	uint8	1
32	Manual operation mode selection state	uint8	1
31	Antenna deployment executed	uint8	1
33	Current main EDC	uint8	1
34	General telemetry state	uint8	1
35	OBDH data last written flash page	uint32	4
39	EPS data last written flash page	uint32	4
43	TTC 0 data last written flash page	uint32	4
47	TTC 1 data last written flash page	uint32	4
51	Antenna data last written flash page	uint32	4
55	EDC data last written flash page	uint32	4
59	Payload X data last written flash page	uint32	4
63	SBCD packets last written flash page	uint32	4
67	Last position determination timestamp	uint32	4
71	Satellite's latitude	int16	2
73	Satellite's longitude	int16	2
75	Satellite's altitude	int16	2
Total	-	-	77

Table 4.5: OBDH data packet.

4.6 Telecommands

The Table 4.10 summarizes all types of packets transmitted or received by satellite that the OBDH can handle, with the ID number, structure and length, and access type of each packet.

Position	Content	Data type	Len. [bytes]
0	System time on which EPS was last read	uint32	4
4	EPS system time	uint32	4
8	EPS μC temperature in Kelvin	uint16	2
10	EPS Circ. and Beacon μ current in mA	uint16	2
12	Last reset cause ID	uint8	1
13	Reset counter	uint16	2
15	Solar panel -Y +X voltage in mV	uint16	2
17	Solar panel -X +Z voltage in mV	uint16	2
19	Solar panel -Z +Y voltage in mV	uint16	2
21	Solar panel +X current in mA	uint16	2
23	Solar panel -X current in mA	uint16	2
25	Solar panel +Y current in mA	uint16	2
27	Solar panel -Y current in mA	uint16	2
29	Solar panel +Z current in mA	uint16	2
31	Solar panel -Z current in mA	uint16	2
33	MPPT 1 duty cycle in %	uint8	1
34	MPPT 2 duty cycle in %	uint8	1
35	MPPT 3 duty cycle in %	uint8	1
36	Total solar panels output voltage after MPPT in mV	uint16	2
38	Main power bus voltage in mV	uint16	2
40	RTD0 temperature in K	uint16	2
42	RTD1 temperature in K	uint16	2
44	RTD2 temperature in K	uint16	2
46	RTD3 temperature in K	uint16	2
48	RTD4 temperature in K	uint16	2
50	RTD5 temperature in K	uint16	2
52	RTD6 temperature in K	uint16	2
54	Battery voltage in mV	uint16	2
56	Battery current in mA	int16	2
58	Battery average current in mA	int16	2
60	Battery accumulated current in mA	uint16	2
62	Battery charge in mAh	uint16	2
64	Battery monitor temperatura in Ke	uint16	2
66	Battery monitor status register	uint8	1
67	Battery monitor protection register	uint8	1
68	Battery monitor cycle counter	uint8	1
69	Battery monitor RAAC in mAh	uint16	2
71	Battery monitor RSAC in mAh	uint16	2
73	Battery monitor RARC in %	uint8	1
74	Battery monitor RSRC in %	uint8	1
75	Battery heater 1 duty cycle in %	uint16	2
77	Battery heater 2 duty cycle in %	uint16	2
79	MPPT 1 mode	uint8	1
80	MPPT 2 mode	uint8	1
81	MPPT 3 mode	uint8	1
82	Battery heater 1 mode	uint8	1
83	Battery heater 2 mode	uint8	1
Total	-	-	84

Table 4.6: EPS data packet.

Position	Content	Data type	Len. [bytes]
0	System time on which TTC was last read	uint32	4
4	TTC system time	uint32	4
8	Reset counter	uint16	2
10	Last reset cause ID	uint8	1
11	μC voltage supply in mV	uint16	2
13	μC current supply in mA	uint16	2
15	μC temperature in Kelvin	uint16	2
17	Input voltage of the radio in mV	uint16	2
19	Input current of the radio in mA	uint16	2
21	Temperature of the radio in K	uint16	2
23	Last valid telecommand (uplink packet ID)	uint8	1
24	RSSI of the last valid telecommand.	uint16	2
26	Temperature of the antenna module in K	uint16	2
28	Antenna status	uint16	2
30	Antenna deployment status	uint8	1
31	Antenna deploymaent hibernation status	uint8	1
32	TX packet counter	uint32	4
36	RX packet counter	uint32	4
Total	-	-	40

Table 4.7: TTC data packet.

Position	Content	Data type	Len. [bytes]
0	System time on which TTC was last read	uint32	4
4	Antenna status code	uint16	2
6	Antenna 1 status	uint8	1
7	Antenna 1 timeout	uint8	1
8	Antenna 1 burning	uint8	1
9	Antenna 2 status	uint8	1
10	Antenna 2 timeout	uint8	1
11	Antenna 2 burning	uint8	1
12	Antenna 3 status	uint8	1
13	Antenna 3 timeout	uint8	1
14	Antenna 3 burning	uint8	1
15	Antenna 4 status	uint8	1
16	Antenna 4 timeout	uint8	1
17	Antenna 4 burning	uint8	1
18	Antenna ignoring switches	uint8	1
19	Antenna independent burn	uint8	1
20	Antenna armed	uint8	1
21	Antenna temperature	uint16	2
Total	-	-	23

Table 4.8: Antenna data packet.

Position	Content	Data type	Len. [bytes]
0	PTT signal receiving time (J2000 epoch)	uint32	4
4	PTT Error code	uint8	1
5	Carrier frequency	int32	4
9	Carrier amplitude at ADC interface output	uint16	2
11	User messsage length in bytes	uint8	1
12	User message	uint8[ ]	1 to 36
Total	-	-	12 to 48

Table 4.9: SBCD PTT data packet.

Link	Packet Name	Payload				Access
		ID	Source Callsign	Data (up to 212 bytes)	Size (bytes)	
Downlink (VHF)	EPS data	00h	" " + "PY0EFS"	EPS data	46	Public
	Message broadcast	01h		Requester + dst. callsign + message	22 to 60	Public
	Ping answer	02h		Requester callsign	15	Public
Downlink (UHF)	General telemetry	10h	" " + "PY0EFS"	OBDH/EPS data	78	Public
	Data request answer	11h		Requester callsign + data ID + ts. + data	20 to 220	Public
	Payload data	12h		Payload ID + payload data	9 to 220	Public
	TC feedback	13h		Req. callsign + TC packet ID + timestamp	20	Public
	Parameter value	14h		Req. callsign + Sub. ID + Param. ID + Param. Val.	21	Public
	Packet broadcast	15h		Data of "Transmit packet" TC	8 to 60	Public
Uplink	Ping request	40h	Any Callsign	None	8	Public
	Data request	41h		Data ID + Start ts. + End ts. + Hash	37	Private
	Broadcast Message	42h		Dst. callsign + message	15 to 53	Public
	Enter hibernation	43h		Hibernation in hours + Hash	30	Private
	Leave hibernation	44h		Hash	28	Private
	Activate module	45h		Module ID + Hash	29	Private
	Deactivate module	46h		Module ID + Hash	29	Private
	Activate payload	47h		Payload ID + Hash	29	Private
	Deactivate payload	48h		Payload ID + Hash	29	Private
	Erase memory	49h		Hash	28	Private
	Force reset	4Ah		Hash	28	Private
	Get payload data	4Bh		Payload ID + Args. + Hash	41	Private
	Set parameter	4Ch		Subsystem ID + Param. ID + Param. value + Hash	34	Private
	Get parameter	4Dh		Subsystem ID + Parameter ID + Hash	30	Private
	Transmit packet	4Eh		Any sequence of bytes + Hash	8 to 60	Private
	Update TLE	4Fh		Line + TLE line + Hash	98	Private

Table 4.10: Telecommunication packets and their content.

The ID of the subsystems, modules, and payloads are available in Table 4.11.

Type	ID Number	Description
Subsystem	0	OBDH
	1	TTC 1
	2	TTC 2
	3	EPS
Module	1	Battery heater
	2	Beacon
	3	Periodic telemetry
Payload	1	EDC 1
	2	EDC 2
	3	Payload X
	4	Radiation instrument
Data	0	OBDH data
	1	EPS data
	2	TTC 0 data
	3	TTC 1 data
	4	Antenna data
	5	SBCD packets
	6	Payload Info

Table 4.11: IDs of the satellite.

4.6.1 Authentication

All the telecommands classified as private use an HMAC authentication scheme. Every type of private telecommand has a unique 16-digit ASCII character key that with the telecommand sequence (or message) generates an 160-bits (20-bytes) hash sequence to be transmitted together with the packet payload. The used hash algorithm is the SHA-1. The Figure 4.2 illustrates this authentication method.

4.6.2 Enter hibernation

This telecommand activates the hibernation mode in a satellite. During the hibernation mode, no transmissions are made by the satellite, it keeps just listening for new incoming packets (reception). The satellite will stay in hibernation mode for a custom period (1 to 65536 minutes), or until a "Leave Hibernation" mode is received. This is a private telecommand, a key is required to send it. Beyond the packet ID and the source callsign (or address), the number o hours (2 bytes long) is also transmitted.

4.6.3 Leave hibernation

This telecommand complements the "enter hibernation" telecommand by deactivating the hibernation mode in the satellite. When a satellite receives this telecommand it enables the transmission again immediately. This is also a private telecommand; a specific key is

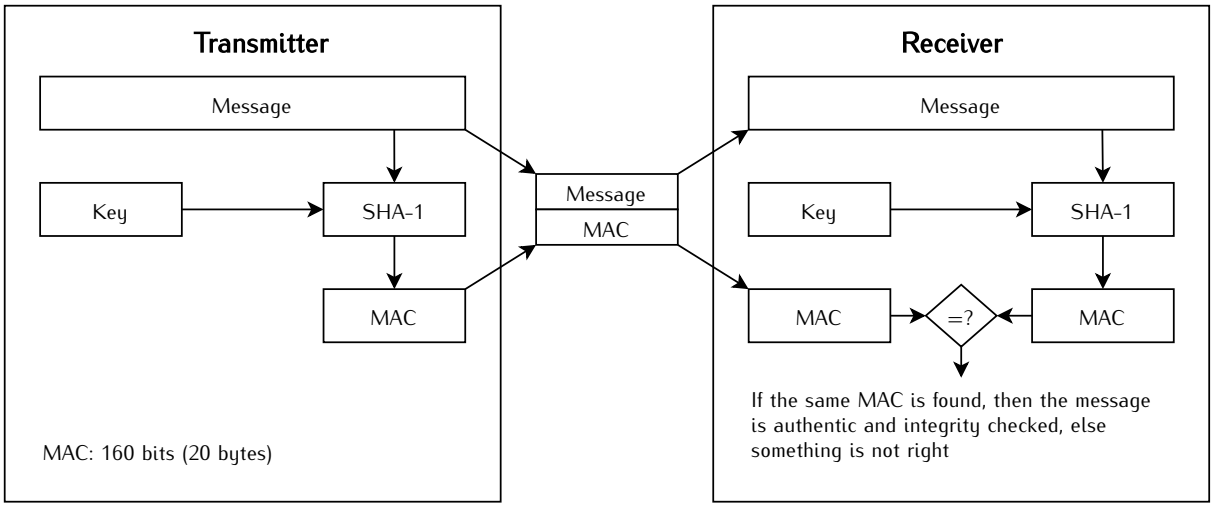


Figure 4.2: Diagram of the used HMAC scheme.

Parameter	Content	Length [bytes]
Packet ID	20h	1
Ground station callsign	Any callsign (ASCII, filled with "0"s)	7
Hibernation period	Period in minutes (1 to 65535)	2
Key	Telecommand key (ASCII)	10
Total	-	20

Table 4.12: Enter hibernation telecommand.

required to send it. There is no additional content to this telecommand packet, just the packet ID and the source callsign (or address).

Parameter	Content	Length [bytes]
Packet ID	21h	1
Ground station callsign	Any callsign (ASCII, filled with "0"s)	7
Key	Telecommand key (ASCII)	10
Total	-	18

Table 4.13: Leave hibernation telecommand.

4.6.4 Activate Module

The "Activate Module" telecommand is a command to activate an internal module of the satellite. Each module has a unique ID that is passed as an argument of this telecommand's packet. The Table 4.14 shows the current used IDs. This is also a private telecommand, and a key is required to transmit it.

Module	ID number
Battery heater	1
Beacon	2
Periodic telemetry	3

Table 4.14: Leave hibernation telecommand.

4.6.5 Deactivate Module

The "Deactivate Module" telecommand complements the telecommand above. It works the same way and with the same parameters, but in this case, it has the purpose of deactivating a given module of the satellite.

4.6.6 Activate Payload

This telecommand is similar to the telecommand "Activate Module", but in this case is used for the activate payloads of the satellite. Each satellite will have a list of IDs of the set of payloads.

This is also a private telecommand, and a key is required to transmit it.

4.6.7 Deactivate Payload

Same as the "Deactivate Module" telecommand, but for payloads.

4.6.8 Erase Memory

The telecommand "erase memory" erases all the content presented in the non-volatile memories of the onboard computer of a satellite. This is a private command, and a key is required to send it. No additional content is required in a erase memory telecommand packet, just the packet ID and the source callsign (or address).

4.6.9 Force Reset

This telecommand performs a general reset of the satellite. When received, the satellite reset all subsystems. This is a private telecommand, and a key is required to send this command to a satellite. There is no additional content in this packet, just the packet ID and the source callsign (or address).

4.6.10 Get Payload Data

This telecommand allows a ground station to download data from a specific satellite payload. The required fields are the payload ID, and, optionally, arguments to be passed to the payload. The IDs and arguments vary according to the satellite. This is a private telecommand, and a key is required to send it.

4.6.11 Set Parameter

This telecommand allows the configuration of specific parameters of a given satellite subsystem. The required fields are the ID of the subsystem to set (1 byte), the ID of the parameter to set (1 byte), and the new value of the parameter (4 bytes long). The possible IDs (subsystem and parameter) vary according to the satellite. This is a private telecommand, and a key is required to send it.

4.6.12 Get Parameter

The telecommand "Get Parameter" complements the "Set Parameter" telecommand. It has the purpose of reading specific parameters of a given subsystem. The required fields are the subsystem's ID (1 byte) and the parameter ID (1 byte). The possible IDs (subsystem and parameter) vary according to the satellite. This is a private telecommand, and a key is required to send it.

4.6.13 Ping

The ping request telecommand is a simple command to test the communication with the satellite. When the satellite receives a ping packet, it will respond with another ping packet (with another packet ID, as defined in the downlink packets list). There are no additional parameters in the ping packet, just the packet ID and the source callsign (or address). It is also a public telecommand; anyone can send a ping request telecommand to a satellite.

Parameter	Content	Length [bytes]
Packet ID	22h	1
Ground station callsign	Any callsign (ASCII, filled with "0"s)	7
Total	-	8

Table 4.15: Ping telecommand.

Parameter	Content	Length [bytes]
Packet ID	12h	1
Satellite callsign	"PY0EGU"	7
Destination callsign	Requester callsign (ASCII, filled with "0"s)	7
Total	-	15

Table 4.16: Ping telecommand answer.

4.6.14 Broadcast message

The "broadcast message" is another public telecommand; no authentication or key is required to send this telecommand to a satellite. This command has the purpose of making a satellite transmit a custom message back to Earth. This can be useful for

communication tasks, like a station sending data to another. There are two parameters in this telecommand: the destination callsign (or address), and the content of the message, which can be any sequence of ASCII characters or any byte value. There is a limit of 38 characters in the message field. The Table 4.17 shows the Telecommand parameters.

Parameter	**Content**	**Length [bytes]**
Packet ID	23h	1
Ground station callsign	Any callsign (ASCII, filled with "0"s)	7
Message	Message to broadcast (ASCII)	up to
Total	-	8

Table 4.17: Message broadcast telecommand.

4.6.15 Data request

The data request telecommand is a command to download data from the satellite. This command allows a ground station to get specific parameters from a given period (stored in the satellite's non-volatile memory of the onboard computer). The list of possible parameters varies according to the satellite. The required fields of this telecommand are the parameter ID (1 byte), the start period in milliseconds (epoch, 4 bytes), and the end period in milliseconds (epoch, 4 bytes). This is a private telecommand, and a key is required to send it.

4.6.16 Update TLE

The Update TLE telecommand is a command to update the Two Line Elements (TLE) lines of the satellite, it sends one line at a time and only updates the lines used for position determination when both lines are update. This is a private telecommand, and a key is required to send it.

4.6.17 Transmit packet

This telecomand is similar to the broadcast message, but it only transmits the package received from the telecommand without any requester or destinations callsigns. This is a private telecommand, and a key is required to send it.

4.7 Operating System

The FreeRTOS 10 [7] is being used as an operating system. FreeRTOS is a market-leading real-time operating system (RTOS) for microcontrollers and small microprocessors. Distributed freely under the MIT open-source license, FreeRTOS includes a kernel and a growing set of IoT libraries suitable for use across all industry sectors. FreeRTOS is built with an emphasis on reliability and ease of use.

The main configuration parameters of the operating system in this project are available in Table 4.18.

Parameter	Value	Unit
Version	v10.2.0	-
Tick rate (Hz)	1000	Hz
CPU clock (HZ)	32	MHz
Max. priorities	5	-
Heap size	40960	bytes
Max. length of task name	20	-

Table 4.18: FreeRTOS main configuration parameters.

More details of the used configuration parameters can be seen in the file *firmware/config/FreeRTOSConfig.h* from [3].

4.8 Hardware Abstraction Layer (HAL)

As the Hardware Abstraction Layer (HAL), the DriverLib [8] from Texas Instruments is begin used. It is the official API to access the registers of the MSP430 microcontrollers.

The DriverLib is meant to provide a "software" layer to the programmer to facilitate a higher programming level than direct register accesses. By using the high-level software APIs provided by DriverLib, users can create powerful and intuitive code that is highly portable between devices within the MSP430 platform and different families in the MSP430/MSP432 platforms.

CHAPTER 5

Board Assembly

The OBDH2 has some DNP components to provide flashing, debugging, testing, or extra interfaces if needed. These components may be optional for the flight model of the board. The draftsman document can be viewed for more detailed information regarding their location and board dimensions [9].

5.1 Development Model

5.1.1 Debug and programming connectors

The P2 and P6 connectors are used for flashing and debugging the OBDH2 board. See again chapter 3 (Hardware) and subsection 3.2.3 (Programmer and Debug) for more information.

5.1.2 Status leds

As already exposed in the document OBDH2 has status LEDs to be used during the development and test phases. See again chapter 2 (System Overview) and subsection 2.3.1 (Status LEDs) for more information.

5.2 Flight Model

The flight model of the OBDH 2.0 boards follows a special assembly, some components are not soldered, like the LEDs and some connectors. The PCB is also fabricated with higher quality, using the Class 3 standard and a core material less sensitive to temperature changes. The silkscreen is also not printed on the board.

5.3 Custom Configuration

On the PC104 connector of OBDH2, there are some jumper resistors to enable extra I2C, SPI, and GPIO interfaces if desired. Note that the I2C0, I2C1, and SPI channels should not be used with shared devices. These components' corresponding tables and locations on the PCB are shown on Table 5.1 and Figures 5.1, 5.2 and 5.3.

Label	Interface
J_PC1	I2C0_SDA
J_PC2	I2C0_SCL
J_PC3	I2C1_SDA
J_PC4	I2C1_SCL
J_PC5	SPI_MOSI
J_PC6	SPI_MISO
J_PC7	SPI_CLK
J_PC8	GPI0
J_PC9	GPIO1
J_PC10	GPIO2
J_PC11	GPIO3

Table 5.1: Additional PC104 inferfaces.

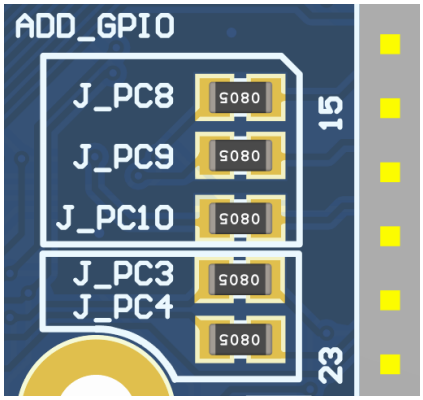


Figure 5.1: Additional GPIOs and I2C channel 1.

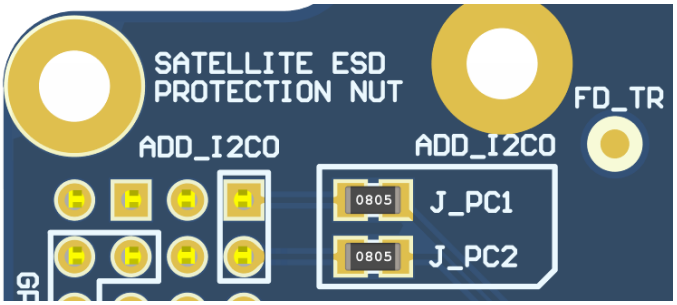


Figure 5.2: Additional I2C channel 0.

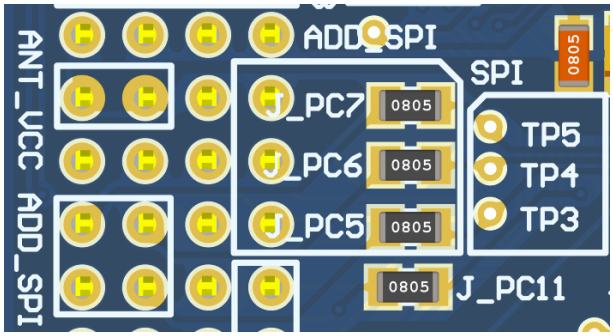


Figure 5.3: Additional GPIO and SPI channel.

CHAPTER 6

Usage Instructions

6.1 Powering the Board

Since the OBDH 2.0 is a service module within a satellite bus, to correctly provide its power supply, it requires an external 3.3 ± 0.2 V power input and a current capability of at least 100 mA (might change depending on the daughterboard requirements). As presented in the PC-104 and programming interface sections, some options are given to power the module to improve flexibility during development. The board has two power schemes: the JTAG interface for debugging, and the PC-104, for the flight configuration. The first case uses both P1 or P2 connectors as power input (besides the JTAG and UART interfaces) and requires a jumper connection in the P6 connector. The second uses the PC-104 pins H1-45 and H1-46 to provide the power, and the P6 connector should remain open. For pinout details, refer to the external connectors in the hardware chapter.

6.2 Log Messages

The OBDH 2.0 has a UART interface dedicated to debugging, described in Table 3.8. It follows a log system structure to improve the information provided in each message. The Figure 6.1 shows an example of the logging system, more specifically the initialization sequence. The messages use the following scheme: in green inside brackets, the timestamp; in magenta, the scope (or origin) of the log; and lastly the actual message, which might be white (info or note), yellow (warning), and red (error).

6.3 Daughterboards Installation

The daughterboard requirements might change for each application board attached. Then, it is important to check at least a minimal set of mandatory characteristics. First, it is important to verify mechanical parameters that concern size (recommended 63.5×43.5 mm), maximum height (no higher than 7 mm), screw attachment (refer to mechanical sheet [9]), and contact connector positioning. After this, the electrical interface must be checked (refer to subsection 3.2.4). There are 3 different power supply options, a 3.3 V source shared with the OBDH board itself, another 3.3 V source shared with the antenna deployer, and the main battery bus that ranges from 5.4 to 8.4 V. Lastly, depending on the application board design, it is necessary to check communication interface protocols and parameters, control inputs and outputs, and external interfaces with other modules.

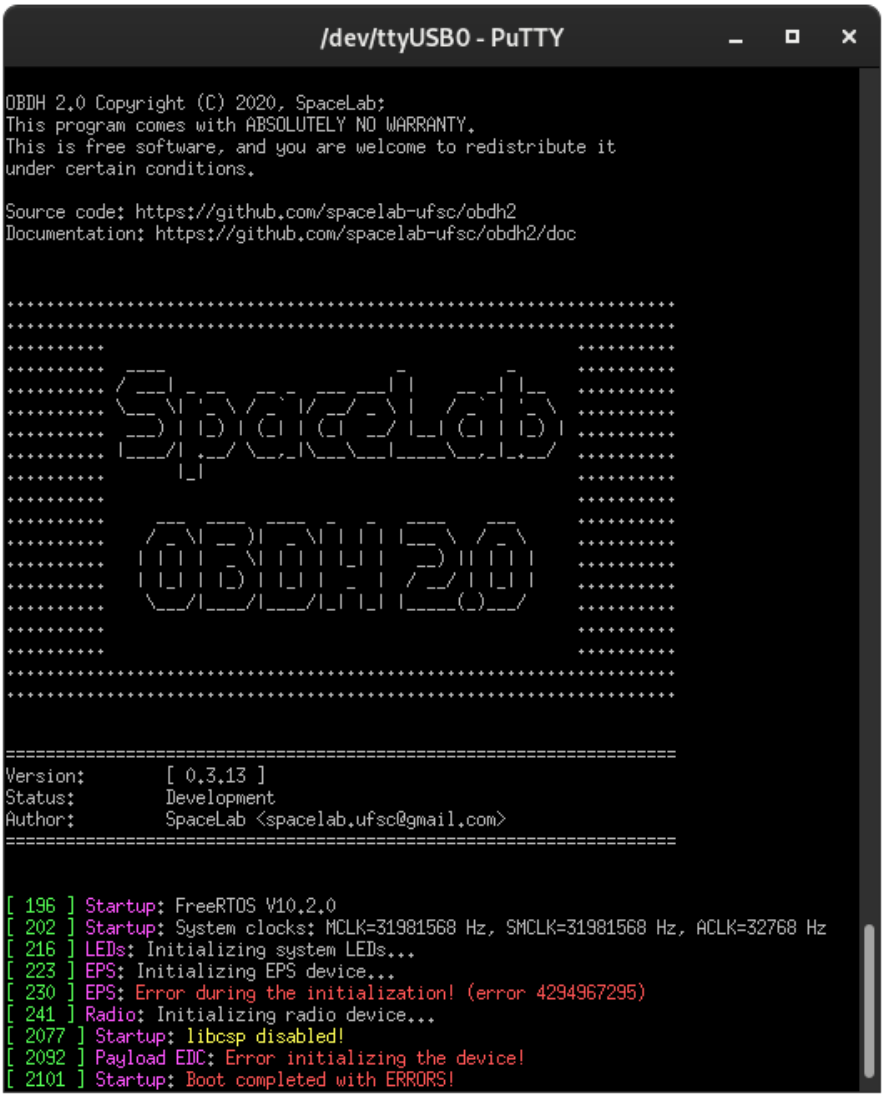


Figure 6.1: Firmware initialization on PuTTy.

Bibliography

[1] SpaceLab. FloripaSat-2 Documentation, 2021. Available at <https://github.com/spacelab-ufsc/floripasat2-doc>.

[2] SpaceLab. On-Board Data Handling, 2019. Available at <https://github.com/floripasat/obdh>.

[3] SpaceLab. On-Board Data Handling 2.0, 2020. Available at <https://github.com/spacelab-ufsc/obdh2>.

[4] Samtec. FSI-110-03-G-D-AD, 2020. Available at <https://www.samtec.com/products/fsi-110-03-g-d-ad>.

[5] Texas Instruments Inc. *MSP430x5xx and MSP430x6xx Family User's Guide*, October 2016.

[6] Texas Instruments Inc. *TPS328x Voltage Monitor With Watchdog Timer*, July 2020.

[7] Amazon Web Services, Inc. FreeRTOS - Real-time operating system for microcontrollers, 2020. Available at <https://www.freertos.org/>.

[8] Texas Instruments. MSP Driver Library, 2020. Available at <https://www.ti.com/tool/MSPDRIVERLIB>.

[9] SpaceLab. On-board data handling 2.0 draftsman document, 2020. Available at <https://github.com/spacelab-ufsc/obdh2/tree/master/hardware/outputs/board_draftsman>.

APPENDIX A

Telecommunication Packets

This appendix lists all the packets that OBDH expects to be received by the RF links: downlink and uplink. The fields and length of each type of packet is also presented.

A.1 Downlink

In Table A.1 the content of the downlink packets is available.

Packet	Position	Content	Length [bytes]
General telemetry	0	Packet ID (20h)	1
	1	Source callsign (" PY0EFS")	7
	8	Time counter in seconds	4
	12	Temperature of the OBDH μC in Kelvin	2
	14	Input current of the OBDH in mA	2
	16	Input voltage of the OBDH in mV	2
	18	Last reset cause of the OBDH	1
	19	Reset counter of the OBDH	2
	21	Last valid telecommand (uplink packet ID)	1
	22	Temperature of the radio in Kelvin	2
	24	RSSI of the last valid telecommand	2
	26	Temperature of the antenna in Kelvin	2
	28	Antenna status	2
	30	Temperature of the EPS μC in K	2
	32	EPS circuitry and Beacon MCU current in mA	2
	34	Last reset cause of the EPS	1
	35	Reset counter (EPS)	2
	37	-Y and +X sides solar panel voltage in mV	2
	39	-X and +Z sides solar panel voltage in mV	2
	41	-Z and +Y sides solar panel voltage in mV	2

	43	-Y side solar panel current in mA	2
	45	+Y side solar panel current in mA	2
	47	-X side solar panel current in mA	2
	49	+X side solar panel current in mA	2
	51	-Z side solar panel current in mA	2
	53	+Z side solar panel current in mA	2
	54	MPPT 1 duty cycle in %	1
	55	MPPT 2 duty cycle in %	1
	56	MPPT 3 duty cycle in %	1
	58	Main power bus voltage in mV	2
	60	Batteries voltage in mV	2
	62	Batteries current in mA	2
	64	Batteries average current in mA	2
	66	Batteries accumulated current in mA	2
	68	Batteries charge in mAh	2
	70	Battery monitor IC temperature in K	2
	72	Battery heater 1 duty cycle in %	1
	74	Battery heater 2 duty cycle in %	1
	75	Main EDC (01h = EDC_1, 02h = EDC_2)	1
	76	Active payload 1 (Payload ID)	1
	77	Active payload 2 (Payload ID)	1
			78
Ping answer	0	Packet ID (21h)	1
	1	Source callsign (“ PY0EFS”)	7
	8	Requester callsign	7
			15
Data request answer	0	Packet ID (22h)	1
	1	Source callsign (“ PY0EFS”)	7
	8	Requester callsign	7
	15	Data type ID	1
	16	Timestamp	4
	20	Data	Var.
			20 (min.)
Message broadcast	0	Packet ID (23h)	1
	1	Source callsign (“ PY0EFS”)	7
	8	Requester callsign	7
	15	Destination callsign	7
	22	Message	up to 38
			218
TC feedback	0	Packet ID (25h)	1
	1	Source callsign (“ PY0EFS”)	7
	8	Requester callsign	7
	15	TC packet ID	1
	16	Timestamp	4

			20
Parameter value	0	Packet ID (26h)	1
	1	Source callsign (“ PY0EFS”)	7
	8	Requester callsign	7
	15	Subsystem ID	1
	16	Parameter ID	1
	17	Parameter value	4
			21

Table A.1: Downlink packets.

A.2 Uplink

As shown in Table 4.10, there are 15 supported telecommands. Below there is a description of each one.

- **Ping Request:** It is a simple command to test the communication with the satellite. When the satellite receives a ping packet, it will respond with another ping packet (with another packet ID, as defined in the downlink packets list). There are no additional parameters in the ping packet, just the packet ID and the source callsign (or address). It is also a public telecommand, anyone can send a ping request telecommand to a satellite.

- **Data Request:** It is a command to download data from the satellite. This command allows a ground station to get specific parameters from a given period (stored in the non-volatile memory of the onboard computer of the satellite). The list of possible parameters varies according to the satellite. The required fields of this telecommand are the parameter ID (1 byte), the start period in milliseconds (epoch, 4 bytes), and the end period in milliseconds (epoch, 4 bytes). This is a private telecommand, and a key is required to send it.

- **Broadcast Message:** The "broadcast message" is another public telecommand, no authentication or key is required to send this telecommand to a satellite. This command has the purpose of making a satellite transmit a custom message back to Earth. This can be useful for communication tasks, like a station sending data to another. There are two parameters in this telecommand: the destination callsign (or address), and the content of the message, which can be any sequence of ASCII characters or any byte value. There is a limit of 38 characters in the message field.

- **Enter Hibernation:** This telecommand activates the hibernation mode in a satellite. During the hibernation mode, no transmissions are made by the satellite; it keeps just listening for new incoming packets (reception). The satellite will stay in hibernation mode for a custom period (1 to 65536 minutes), or until a "Leave Hibernation" mode is received. This is a private telecommand, a key is required to send it. Beyond the packet ID and the source callsign (or address), the number o minutes (2 bytes long) is also transmitted.

- **Leave Hibernation:** This telecommand complements the "enter hibernation" telecommand by deactivating the hibernation mode in the satellite. When a satellite receives this telecommand, it enables the transmission again immediately. This is also a private telecommand; a specific key is required to send it. There is no additional content to this telecommand packet, just the packet ID and the source callsign (or address).

- **Activate Module:** It activates an internal module of the satellite. Each module has a unique ID that is passed as an argument of this telecommand's packet. The module Battery heater's ID is 1, Beacon's ID is 2 and Periodic telemetry has ID number 3.

- **Activate Payload:** This one is similar to the telecommand "Activate Module", but in this case is used for activating payloads of the satellite. Each satellite will have a list of IDs of the set of payloads. This is also a private telecommand, and a key is required to transmit it.

- **Deactivate Payload:** It is the same as the "Deactivate Module" telecommand, but for payloads.

- **Erase Memory:** It erases all the content presented in the non-volatile memories of the onboard computer of a satellite. This is a private command, and a key is required to send it. No additional content is required in a erase memory telecommand packet, just the packet ID and the source callsign (or address).

- **Force Reset:** It performs a general reset of the OBDH. When received, the satellite reset the OBDH subsystem. This is a private telecommand, and a key is required to send this command to a satellite. There is no additional content in this packet, just the packet ID and the source callsign (or address).

- **Get Payload Data:** It allows a ground station to download data from a specific payload of the satellite. The required fields are the payload ID, and optionally, arguments to be passed to the payload. The IDs and arguments vary according to the satellite. This is a private telecommand, and a key is required to send it.

- **Get Parameter:** It allows the configuration of specific parameters of a given subsystem of the satellite. The required fields are the ID of the subsystem to set (1 byte), the ID of the parameter to set (1 byte), and the new value of the parameter (4 bytes long). The possible IDs (subsystem and parameter) vary according to the satellite. This is a private telecommand, and a key is required to send it.

- **Set Parameter:** This telecommand complements the "Set Parameter" telecommand. It has the purpose of reading specific parameters of a given subsystem. The required fields are the subsystem's ID (1 byte) and the parameter ID (1 byte). The possible IDs (subsystem and parameter) vary according to the satellite. This is a private telecommand, and a key is required to send it.

- **Update TLE** This telecommand provides a way to update the satellite TLE lines, which are used for position determination. This is a private telecommand, and a key is required to send it.

The Table A.2 presents the content of the uplink packets.

Packet	Position	Content	Length [bytes]
Ping request	0	Packet ID (40h)	1
	1	Ground station callsign	7
			8
Data request	0	Packet ID (41h)	1
	1	Ground station callsign	7
	8	Data type ID	1
	9	Start timestamp	4
	13	End timestamp	4
	17	HMAC hash	20
			37
Broadcast message	0	Packet ID (42h)	1
	1	Ground station callsign	7
	8	Destination callsign	7
	15	Message	up to 38
			up to 53
Enter hibernation	0	Packet ID (43h)	1
	1	Ground station callsign	7
	8	Hibernation in hours	2
	10	HMAC hash	20
			30
Leave hibernation	0	Packet ID (44h)	1
	1	Ground station callsign	7
	8	HMAC hash	20
			28
Activate module	0	Packet ID (45h)	1
	1	Ground station callsign	7
	8	Module ID	1
	9	HMAC hash	20
			29
Deactivate module	0	Packet ID (46h)	1
	1	Ground station callsign	7
	8	Module ID	1
	9	HMAC hash	20
			29
Activate payload	0	Packet ID (47h)	1
	1	Ground station callsign	7
	8	Payload ID	1
	9	HMAC hash	20
			29
Deactivate payload	0	Packet ID (48h)	1
	1	Ground station callsign	7
	8	Payload ID	1
	9	HMAC hash	20

			29
Erase memory	0	Packet ID (49h)	1
	1	Ground station callsign	7
	8	HMAC hash	20
			28
Force reset	0	Packet ID (4Ah)	1
	1	Ground station callsign	7
	8	HMAC hash	20
			28
Get payload data	0	Packet ID (4Bh)	1
	1	Ground station callsign	7
	8	Payload ID	1
	9	Payload arguments	12
	21	HMAC hash	20
			41
Set parameter	0	Packet ID (4Ch)	1
	1	Ground station callsign	7
	8	Subsystem ID	1
	9	Parameter ID	1
	10	Parameter value	4
	14	HMAC hash	20
			34
Get parameter	0	Packet ID (4Dh)	1
	1	Ground station callsign	7
	8	Subsystem ID	1
	9	Parameter ID	1
	10	HMAC hash	20
	0	Packet ID (4Eh)	1
	1	Ground station callsign	7
Update TLE	8	Line	1
	9	TLE line	69
	78	HMAC hash	20
			98

Table A.2: Uplink packets.

APPENDIX B

Test Report of v0.5 Version

This appendix is a test report of the first manufactured and assembled PCB (version v0.5).

- **PCB manufacturer**: PCBWay (China)
- **PCB assembly**: PCBWay (China)
- **PCB arrival date**: 2021/04/14
- **Execution date**: 2021/04/16 to TBC
- **Tester**: G. M. Marcelino

B.1 Visual Inspection

- **Test description/Objective**: Inspection of the board, visually and with a multimeter, searching for fabrication and assembly failures.
- **Material**:
  - Multimeter UNI-T DT830B
- **Results**: The results of this test can be seen in Figures B.1 (top view of the board) and B.2 (bottom view of the board).
- **Conclusion**: No problems were identified on this test.

B.2 Firmware Programming

- **Test description/Objective**: Inspection of the board, visually and with a multimeter, searching for fabrication and assembly mistakes.
- **Material**:
  - Code Composer Studio v9.3.0
  - MSP-FET Flash Emulation Tool
  - USB-UART converter

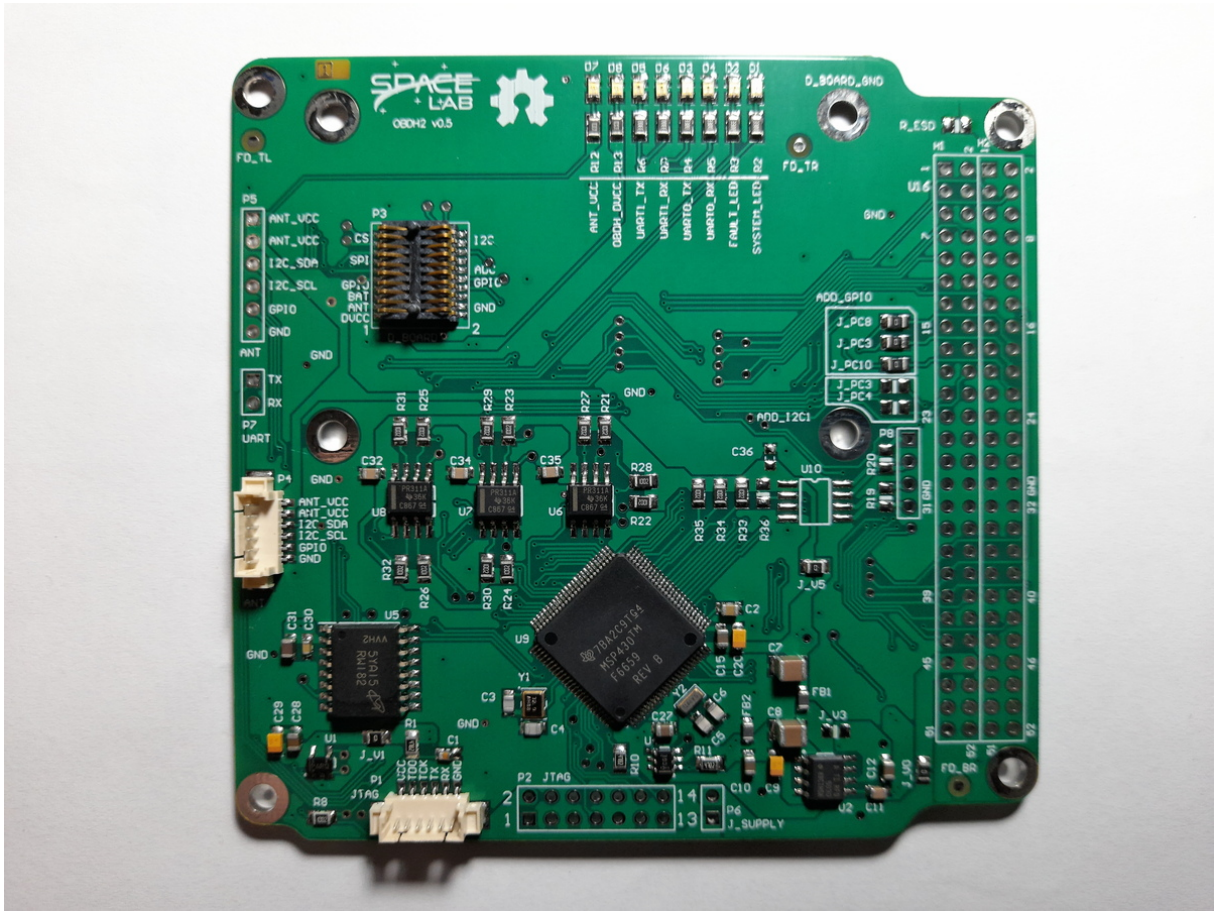


Figure B.1: Top view of the OBDH 2.0 v0.5 board.

  - Screen (Linux software)

- **Results**: The results of this are available in Figure B.3, where the log messages of the first boot of the board can be seen.
- **Conclusion**: No problems were identified on this test.

B.3 Communication Busses

- **Test description/Objective**: Test the communication busses of the board, as listed below:
  - I^2C Port 0
  - I^2C Port 1
  - I^2C Port 2
- **Material**:
  - Saleae Logic Analyzer (24 MHz, 8 channels)
  - Saleae Logic software (v1.2.18)

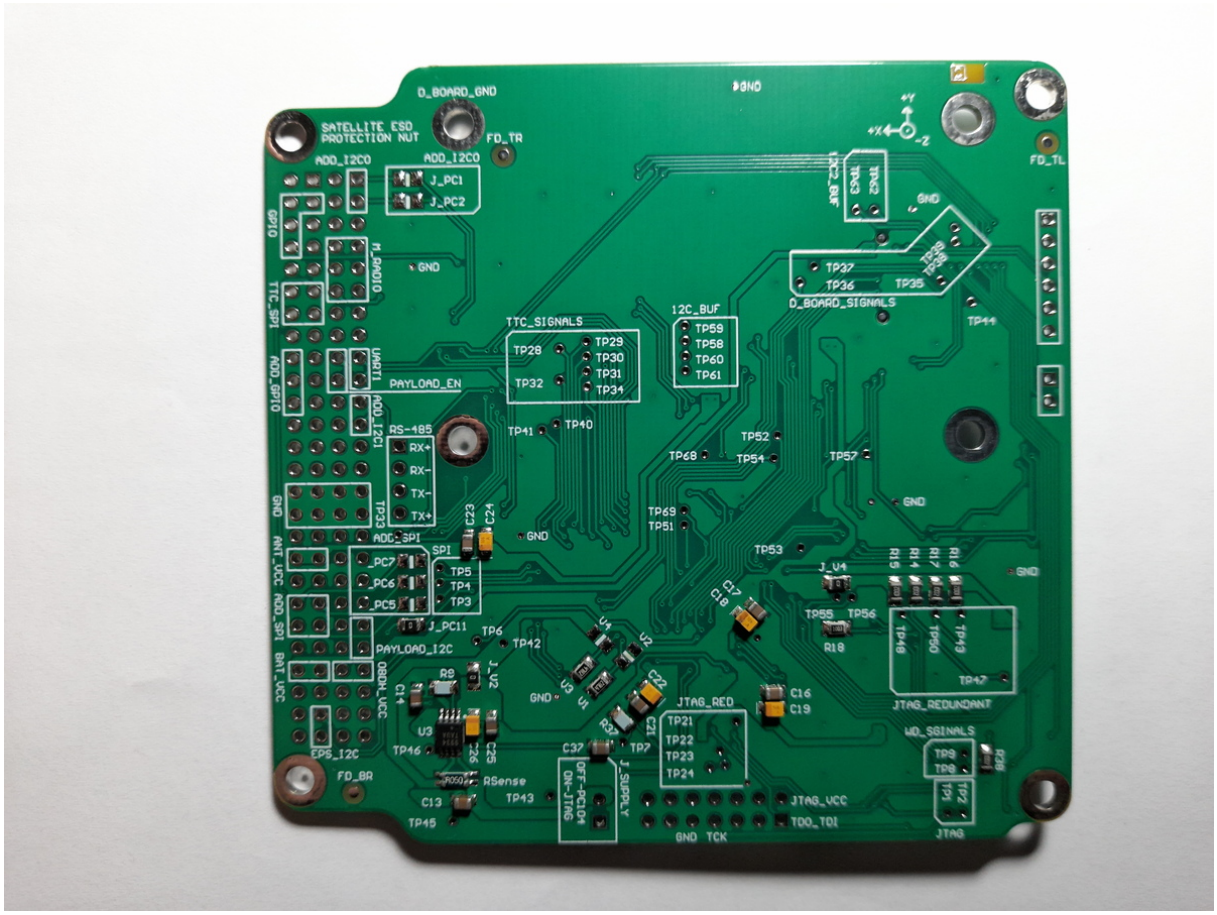


Figure B.2: Bottom view of the OBDH 2.0 v0.5 board.

  - MSP-FET Flash Emulation Tool

- **Results**: The results of this test can be seen in Figures B.4, B.5 and B.6.
- **Conclusion:** No problems were identified on this test, all buses are working as expected.

B.4 Sensors

B.4.1 Input Voltage

- **Test description/Objective**: .
- **Material**:
  - Code Composer Studio v9.3.0
  - MSP-FET Flash Emulation Tool
  - USB-UART converter
  - Screen (Linux software)

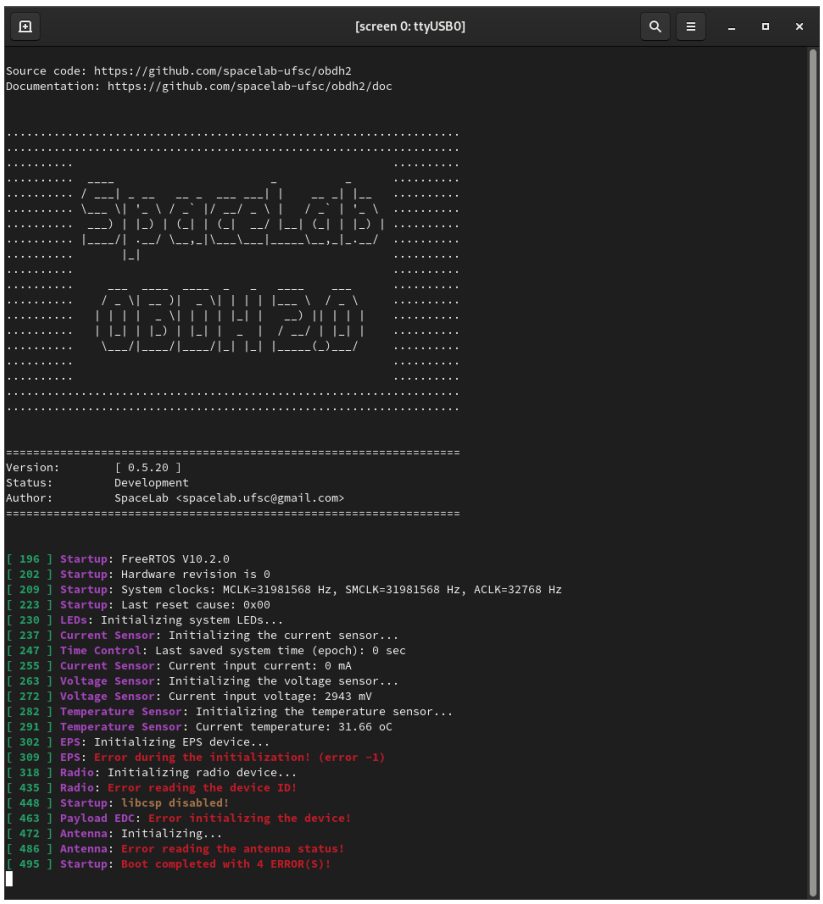
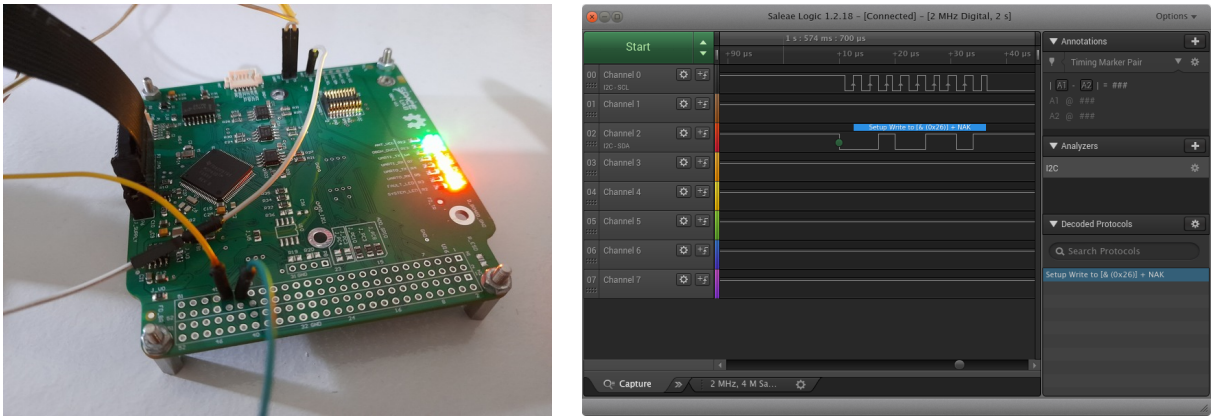


Figure B.3: Log messages during the first boot.

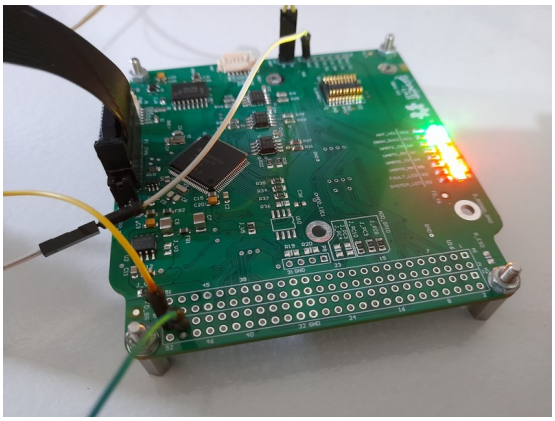


(a) Connections of the I^{2}C port 0 test.

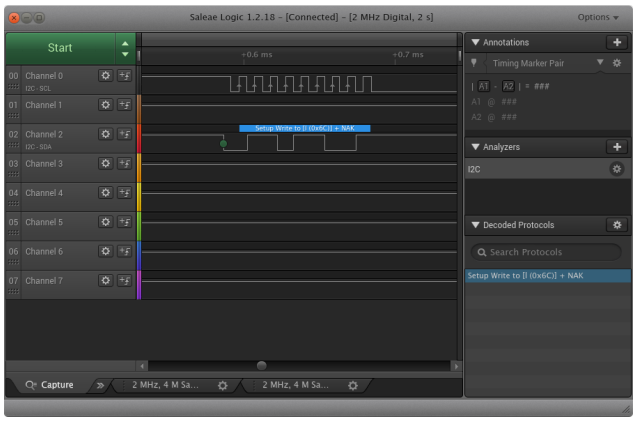
(b) Waveforms of the I^{2}C port 0 test.

Figure B.4: I^{2}C port 0 test.

- **Results**: .
- **Conclusion:** .

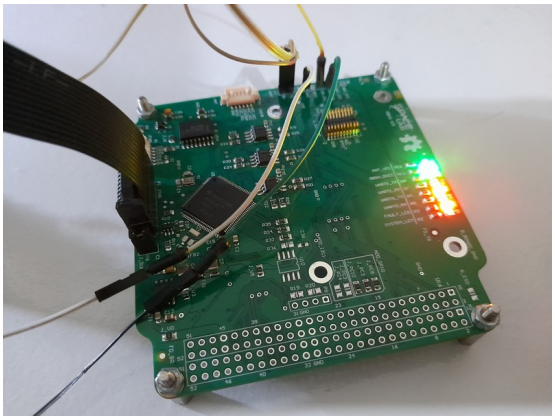


(a) Connections of the I^{2}C port 1 test.

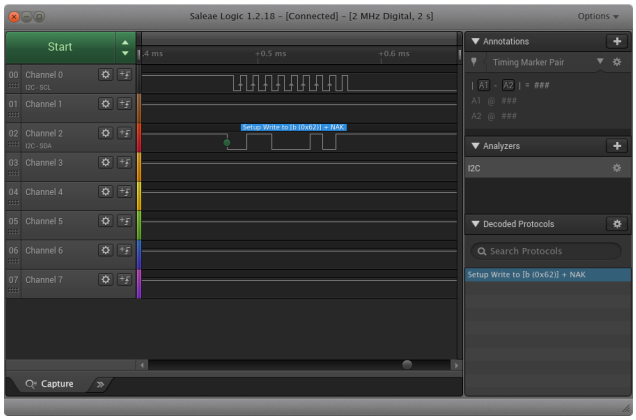


(b) Waveforms of the I^{2}C port 1 test.

Figure B.5: I^{2}C port 1 test.



(a) Connections of the I^{2}C port 2 test.

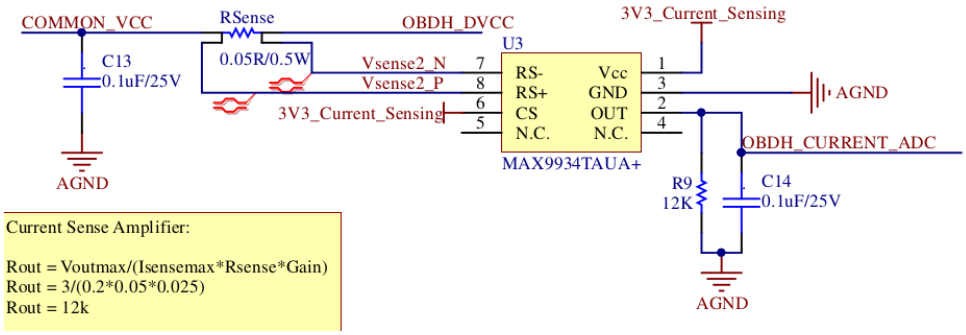


(b) Waveforms of the I^{2}C port 2 test.

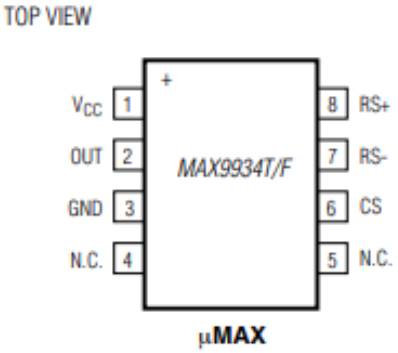
Figure B.6: I^{2}C port 2 test.

B.4.2 Input Current

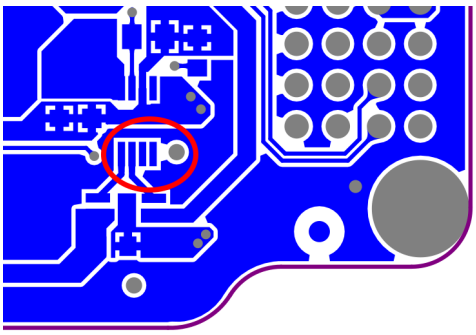
- **Test description/Objective**: .
- **Material**:
  - Code Composer Studio v9.3.0
  - MSP-FET Flash Emulation Tool
  - USB-UART converter
  - Screen (Linux software)
- **Results**: .
- **Conclusion:** .



(a) Current sensing circuit.



(b) MAX9934 pinout.



(c) Current sensing layout (bottom layer).

Figure B.7: .

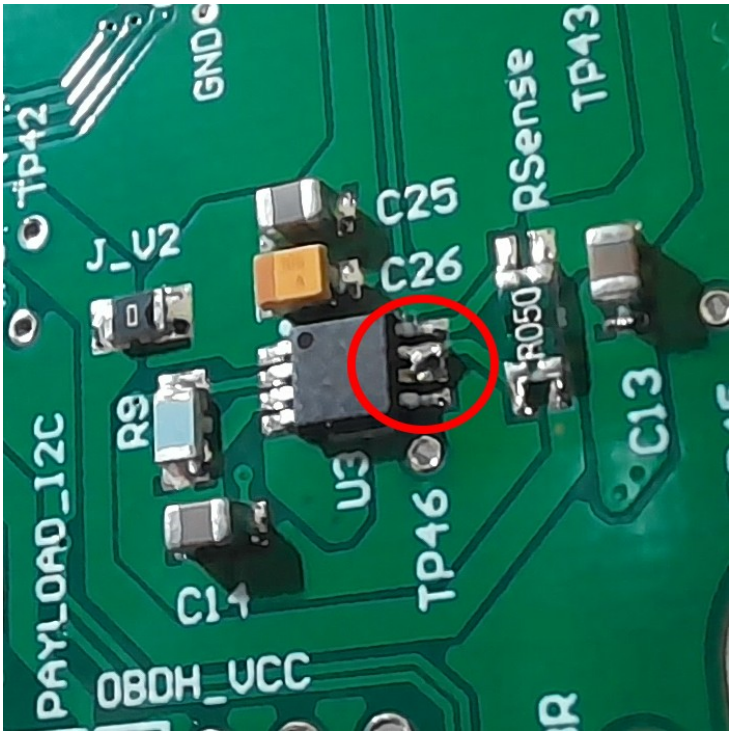


Figure B.8: Current sensor fix.

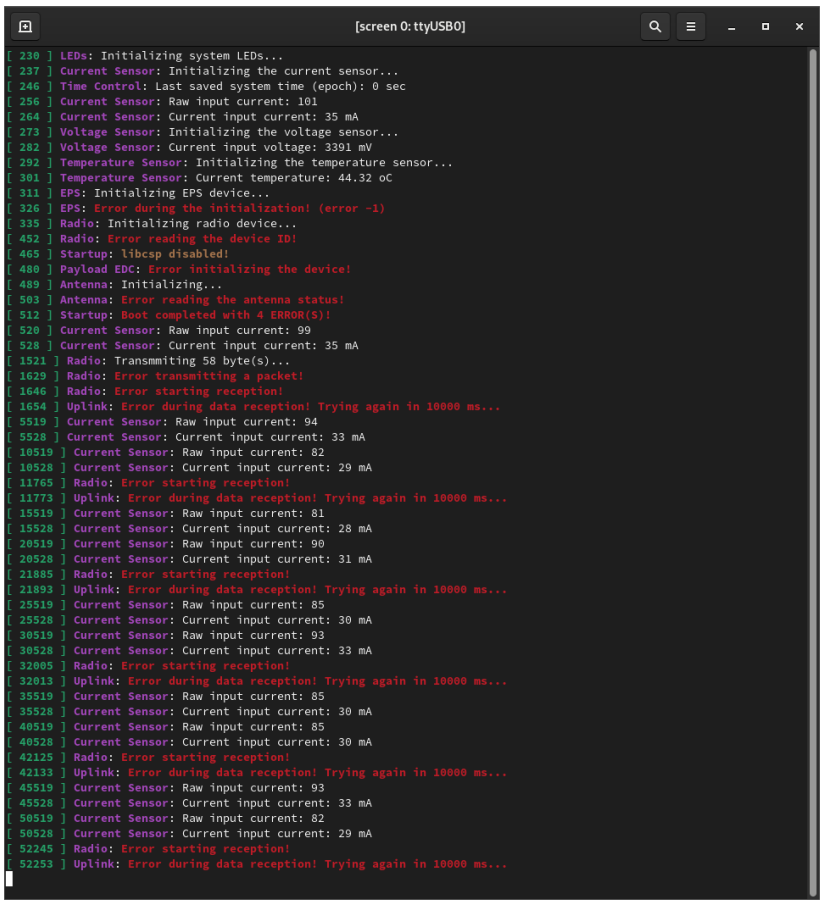
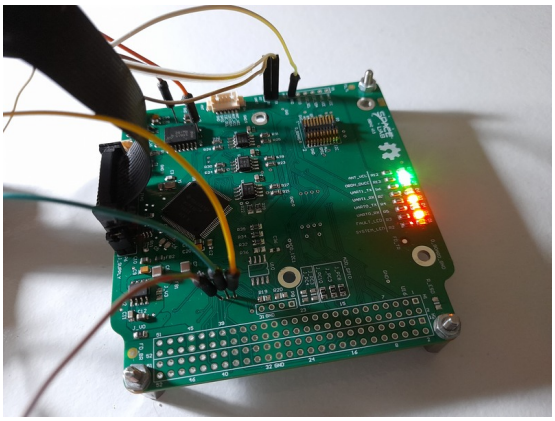


Figure B.9: Log messages with the read values from the current sensor.

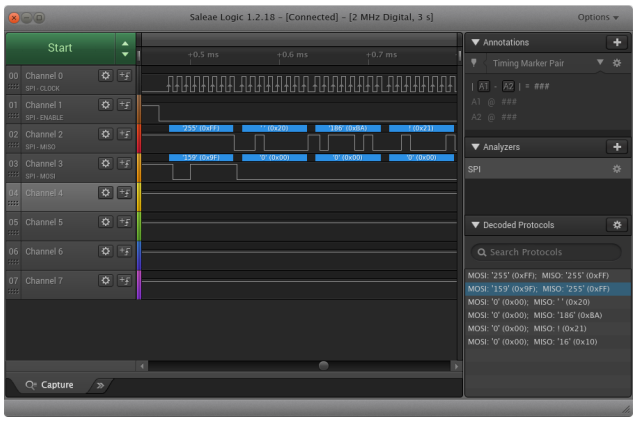
B.5 Peripherals

B.5.1 NOR Flash Memory

- **Test description/Objective**: Test the functionality of the NOR flash memory by verifying the device ID register of the IC.
- **Material**:
  - Saleae Logic Analyzer (24 MHz, 8 channels)
  - Saleae Logic software (v1.2.18)
  - MSP-FET Flash Emulation Tool
- **Results**: The results of this test can be seen in Figure B.10.
- **Conclusion:** No problems were identified on this test, as can be seen in Figure B.10(b), the device ID register was read as expected.



(a) Connections of the NOR flash memory test.



(b) Waveforms of the NOR memory SPI.

Figure B.10: NOR memory SPI test.

B.6 Conclusion

Excluding the current sensor issue, no major problems were identified during the executed tests. For the next fabrication round, the identified mistakes will be corrected.

APPENDIX C

Test Report of v0.7 Version

This appendix is a test report of the first manufactured and assembled PCB (version v0.7).

- **PCB manufacturer**: PCBWay (China)
- **PCB assembly**: PCBWay (China)
- **PCB arrival date**: 2022/04/18
- **Execution date**: 2022/04/22 to 2022/08/10
- **Tester**: Gabriel M. Marcelino, Vitória B. Bianchin and Bruno Benedetti
- **DNP components**: P8, P2, P5, P6, P7, D1, D2, D3, D4, D5, D6, D7, D8, U10, R19, R20, R_ESD, J_PC3, J_PC4, R2, R3, R4, R5, R6, R7, R12, R13, J_V3, J_PC5, J_PC6, J_PC7, J_PC1, J_PC2, V1, V4, R36, C36

C.1 Visual Inspection

- **Test description/Objective**: Inspection of the board, visually and with a multimeter, searching for fabrication and assembly failures.
- **Material**:
  - Digital microscope (1000x)
  - Multimeter Fluke 17B+
- **Results**: The results of this test can be seen in Figures C.1 (top view of the board) and C.2 (bottom view of the board).
- **Conclusion**: No problems were identified on this test.

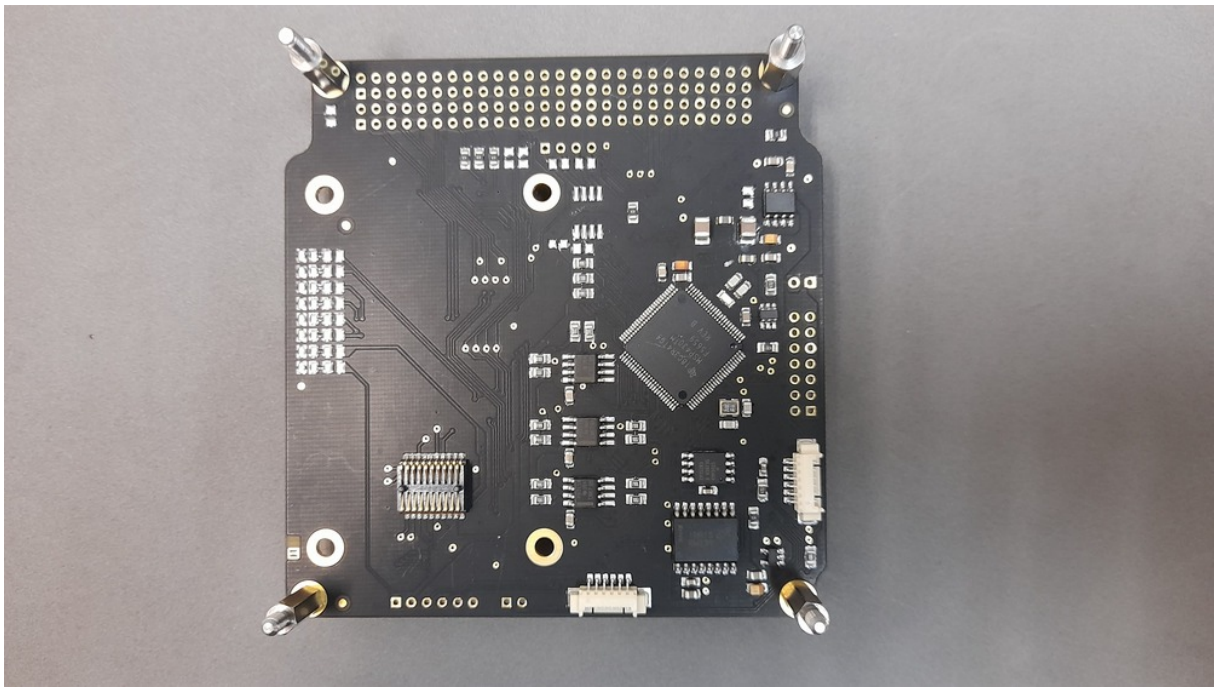


Figure C.1: Top view of the OBDH 2.0 v0.7 board.

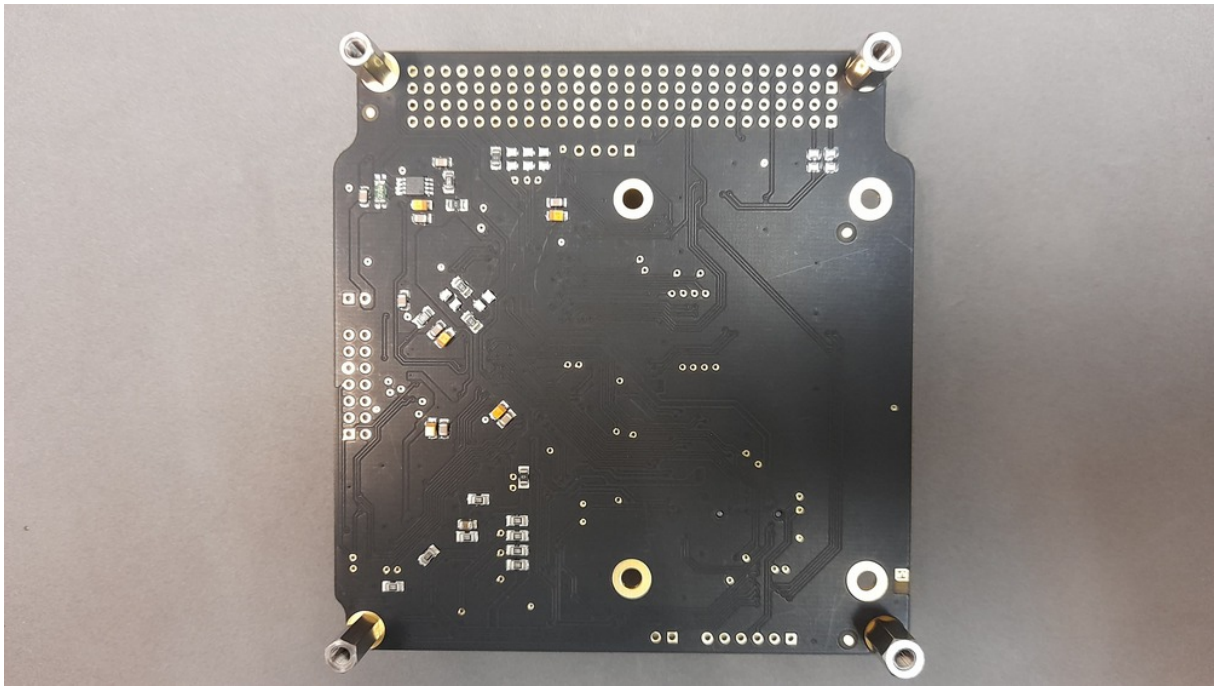


Figure C.2: Bottom view of the OBDH 2.0 v0.7 board.

C.2 Firmware Programming

- **Test description/Objective**: Inspection of the board, visually and with a multimeter, searching for fabrication and assembly mistakes.
- **Material**:

  - Code Composer Studio v11
  - MSP-FET Flash Emulation Tool
  - USB-UART converter
  - PuTTy

- **Results**: The results of this are available in Figure C.3, where the log messages of the first boot of the board can be seen.
- **Conclusion**: No problems were identified on this test.

Figure C.3: Log messages during the first boot.

C.3 Communication Busses

- **Test description/Objective**: Test the communication busses of the board, as listed below:
  - I^2C Port 0
  - I^2C Port 1

  - I^2C Port 2

- **Material**:
  - Saleae Logic Analyzer (24 MHz, 8 channels)
  - Saleae Logic software (v2)
  - MSP-FET Flash Emulation Tool
- **Results**: The results of this test can be seen in Figures C.5, C.6 and C.7.
- **Conclusion:** No problems were identified on this test, all buses are working as expected.

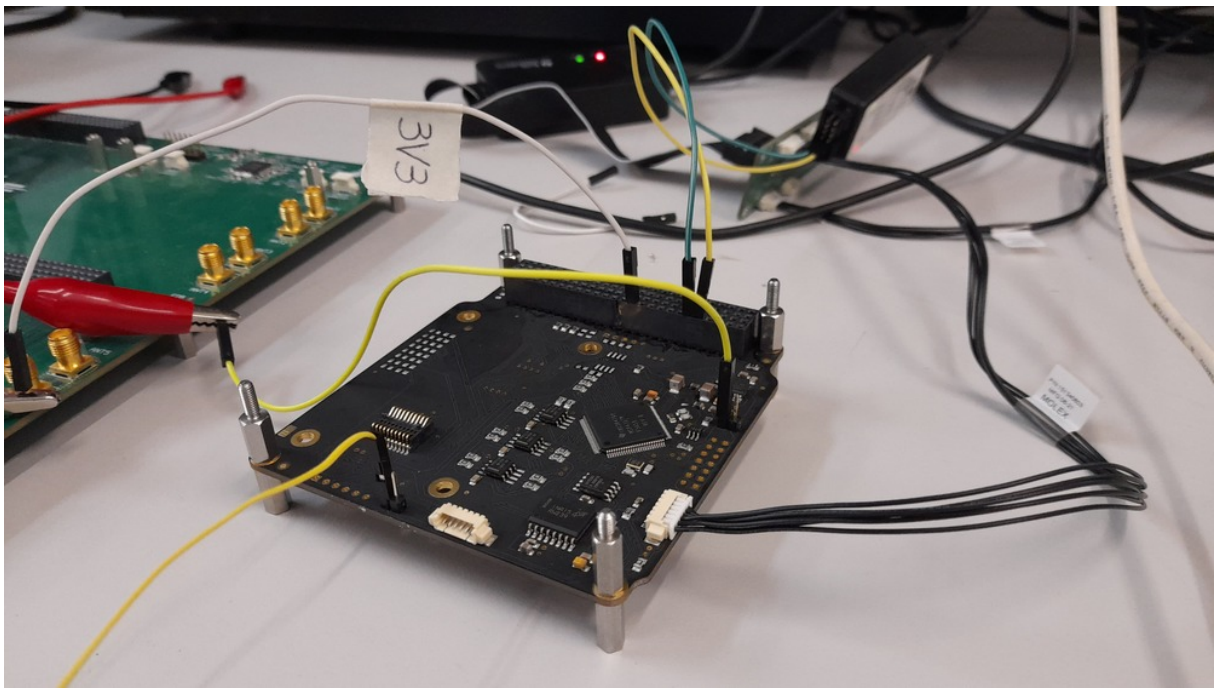


Figure C.4: Setup of the I2C port tests.

C.4 Sensors

C.4.1 Input Voltage

- **Test description/Objective**: Verify the input voltage measurements of the board.
- **Material**:
  - Code Composer Studio v11
  - MSP-FET Flash Emulation Tool
  - Programmable power supply
  - USB-UART converter

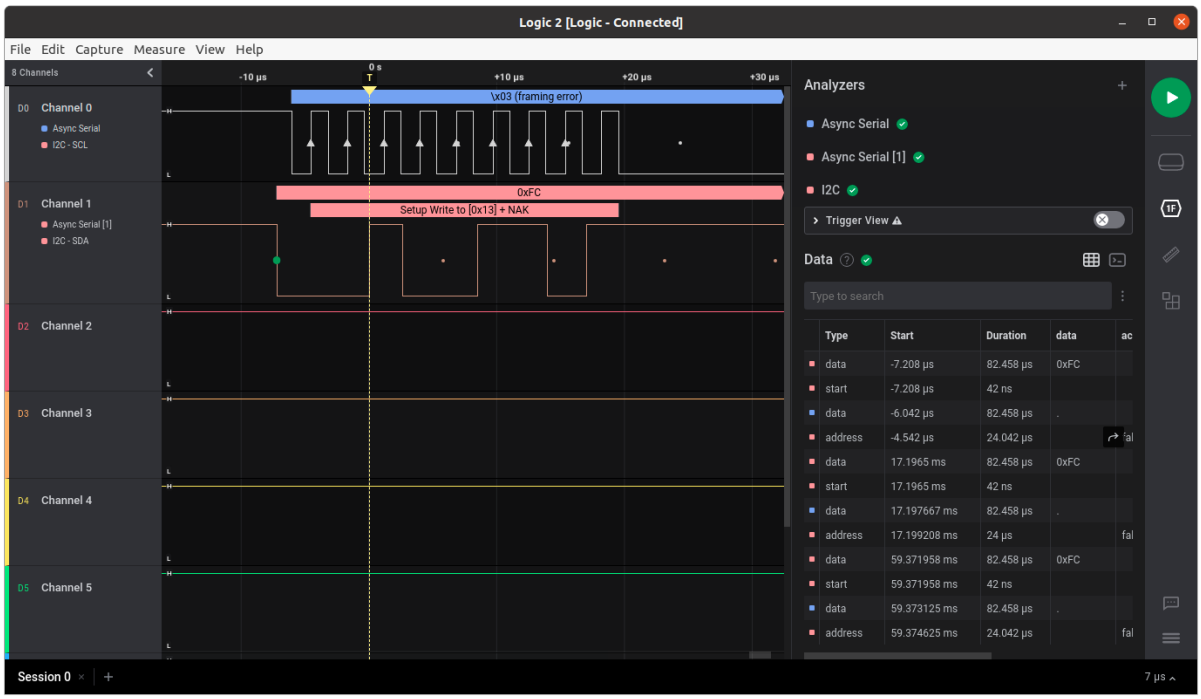


Figure C.5: Waveform of the I2C port 0.

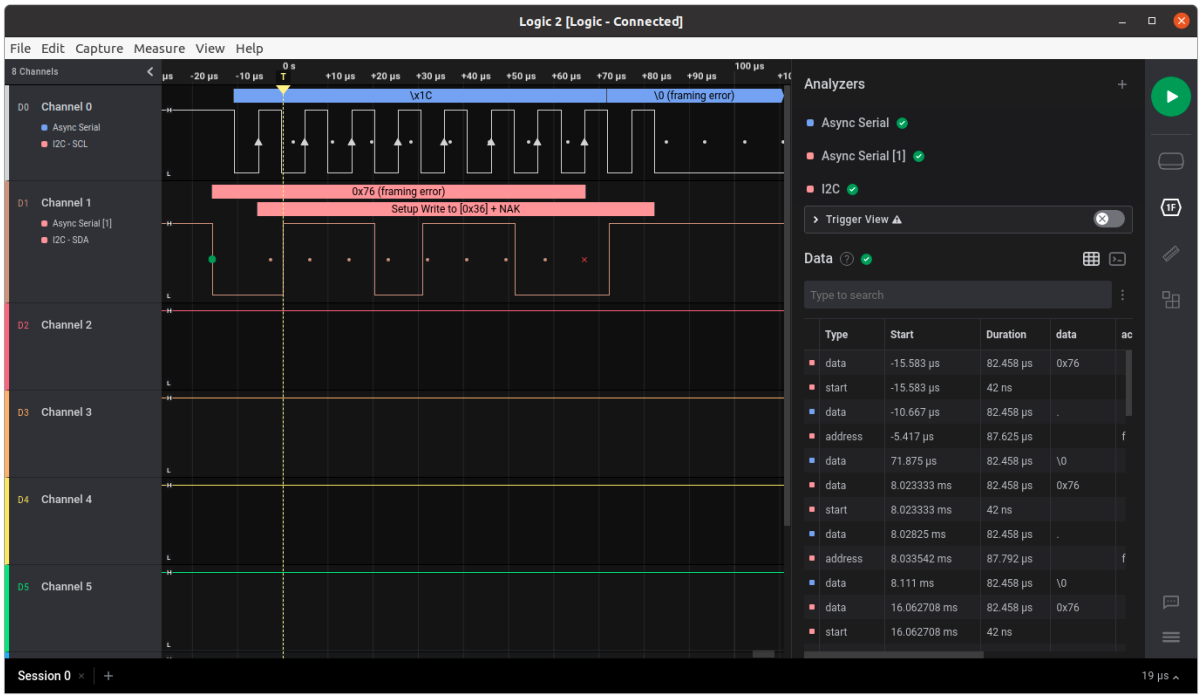


Figure C.6: Waveform of the I2C port 1.

  - Screen (Linux software)

- **Results**: TBC.
- **Conclusion:** The input voltage was measured correctly by the sensor.

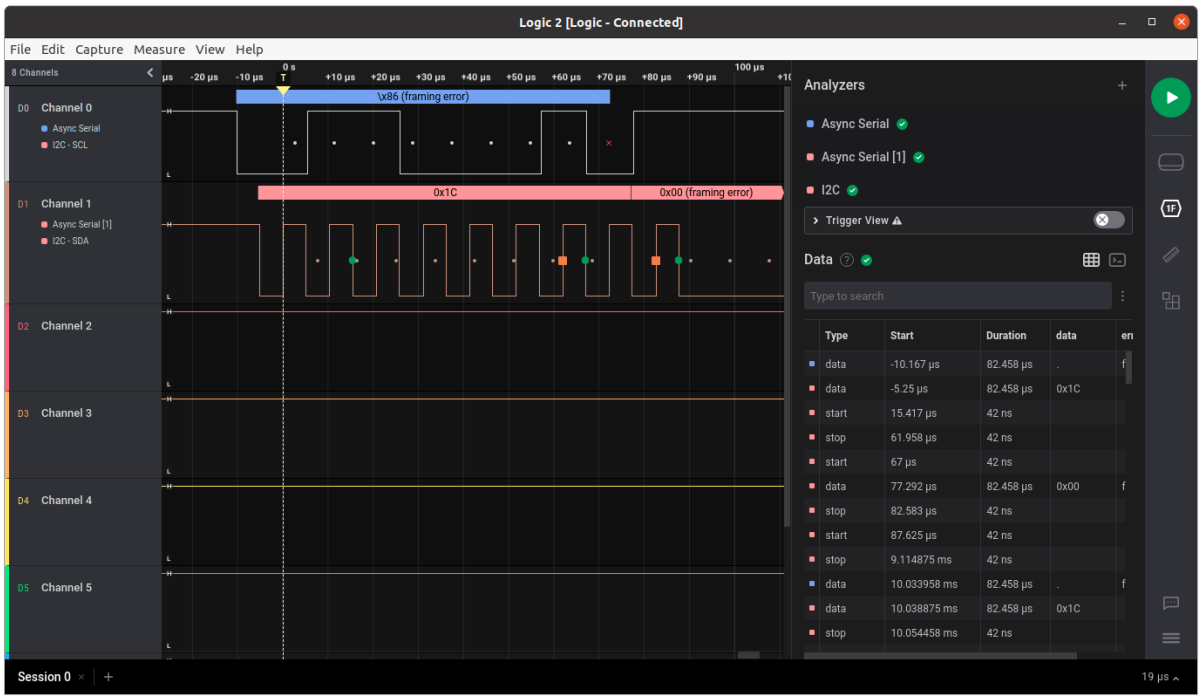


Figure C.7: Waveform of the I2C port 2.

C.4.2 Input Current

- **Test description/Objective**: Verify the input current measurements of the board.
- **Material**:
  - Code Composer Studio v11
  - MSP-FET Flash Emulation Tool
  - Programmable power supply
  - USB-UART converter
  - Screen (Linux software)
- **Results**: TBC.
- **Conclusion:** The input current was measured correctly by the sensor.

C.5 Peripherals

C.5.1 NOR Flash Memory

- **Test description/Objective**: Test the functionality of the NOR flash memory by verifying the device ID register of the IC and performing writing/reading operations.
- **Material**:
  - Code Composer Studio v11

  - MSP-FET Flash Emulation Tool
  - USB-UART converter
  - Screen (Linux software)

- **Results**: The results of this test can be seen in Figure C.8.
- **Conclusion:** No problems were identified on this test, as can be seen in Figure C.8, an writing/reading operation were executed with success.

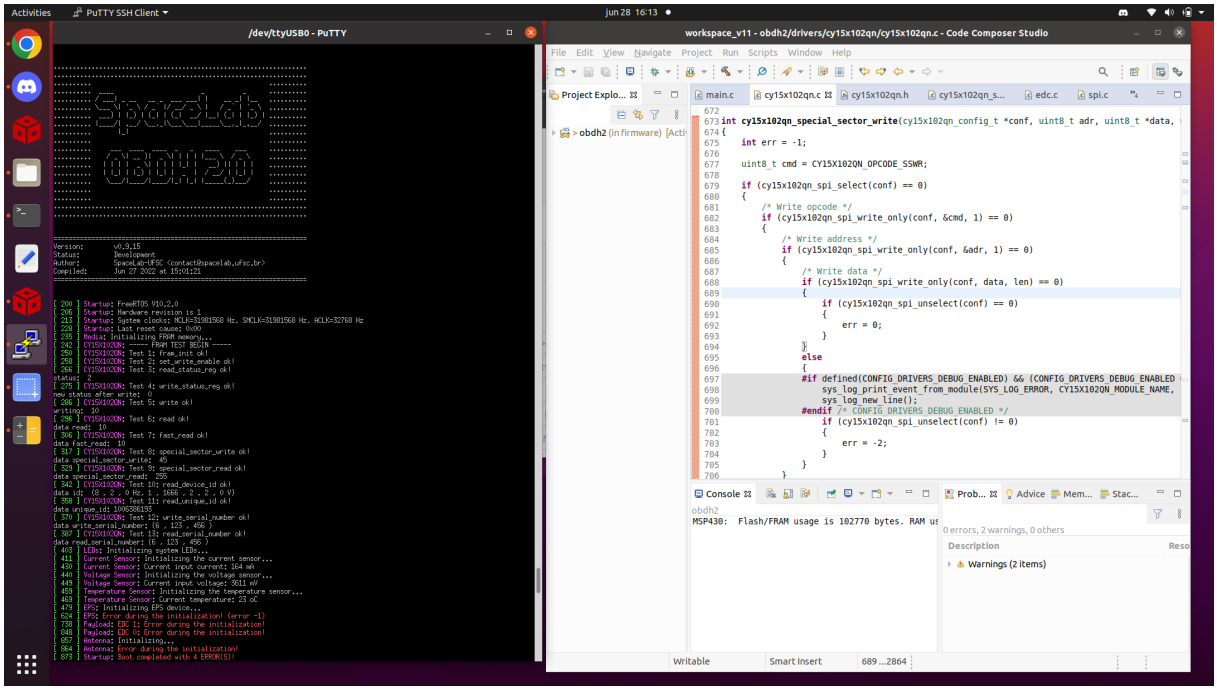
Figure C.8: Test results of the NOR flash memory.

C.5.2 FRAM Memory

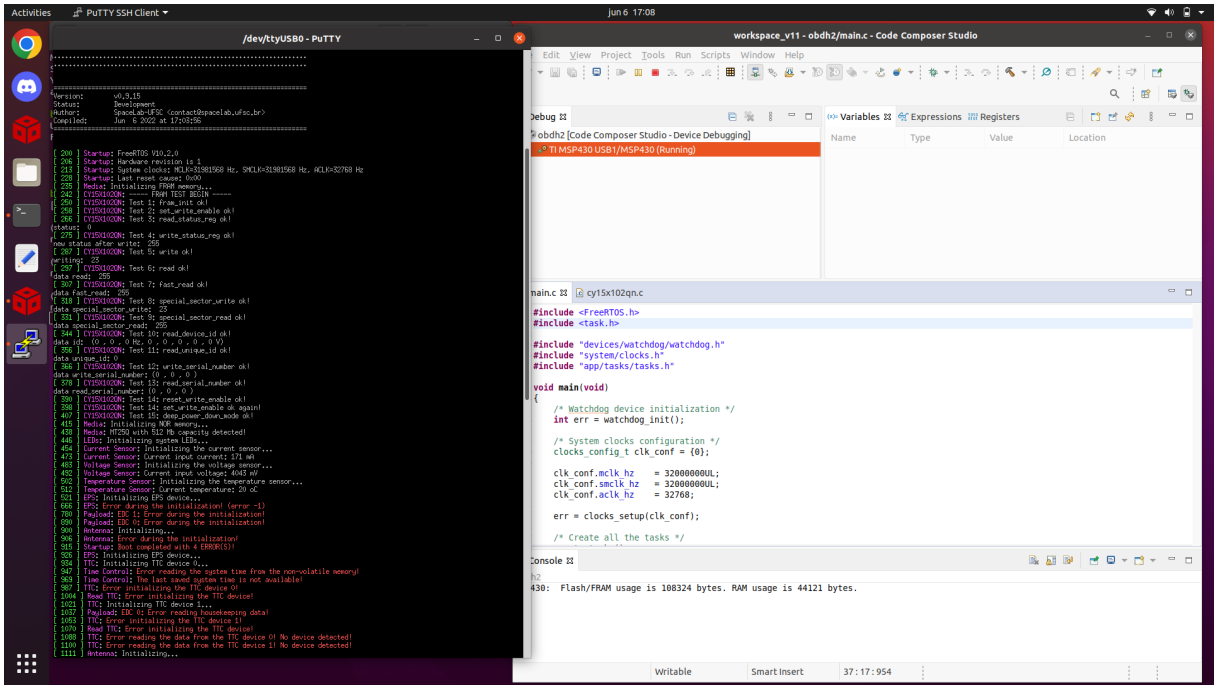
- **Test description/Objective**: Test the functionality of the FRAM memory by verifying the device ID register of the IC and performing writing/reading operations.
- **Material**:
  - Code Composer Studio v11
  - MSP-FET Flash Emulation Tool
  - USB-UART converter
  - Screen (Linux software)
- **Results**: The results of this test can be seen in Figure C.9.
- **Conclusion:** No problems were identified on this test.

C.6 Conclusion

No major problems were identified during the executed tests, all peripherals all working as expected.



(a)



(b)

Figure C.9: Test results of the FRAM memory.